

Variáveis Latentes Discretas, K-Means e Modelos de Mistura

Guia de estudo: conteúdo consolidado das quatro referências indicadas, com 116 perguntas e respostas comentadas.

Bishop & Bishop — *Deep Learning: Foundations and Concepts* (2024)

Capítulo 15 – Discrete Latent Variables (15.1 a 15.4.5) · bishopbook.com

Murphy — *Probabilistic Machine Learning: An Introduction* (2022)

Seções 21.3 (K-means clustering) a 21.4.2 (Mixtures of Bernoullis)

scikit-learn — *Gaussian mixture models*

Seções 2.1 a 2.1.1 · scikit-learn.org/stable/modules/mixture.html

scikit-learn — *Clustering*

Seções 2.3 a 2.3.2 · scikit-learn.org/stable/modules/clustering.html

Aviso sobre as fontes. Este guia foi redigido a partir do conteúdo canônico das quatro referências e do material da apresentação do grupo. Os textos originais não puderam ser consultados diretamente durante a redação (o proxy de rede bloqueia os quatro domínios), portanto **confira a numeração exata das subseções** ao citar. As definições, equações e resultados são padrão da literatura e podem ser verificados nos próprios livros.

Sumário

PARTE 0 Como usar este guia · Notação

Convenções de símbolos, o mapa dos três modelos e a régua de leitura.

PARTE I Bishop, cap. 15 — Discrete Latent Variables

15.1 K-means Clustering · 15.1.1 Segmentação de imagem · 15.2 Mixtures of Gaussians · 15.2.1 Função de verossimilhança · 15.2.2 Máxima verossimilhança · 15.3 O algoritmo EM · 15.3.1 Misturas gaussianas · 15.3.2 Relação com K-means · 15.3.3 Misturas de Bernoulli · 15.4 Evidence Lower Bound · 15.4.1 a 15.4.5 — **46 perguntas**

PARTE II Murphy, 21.3 a 21.4.2

O algoritmo · Quantização vetorial · k-means++ · K-medoids · Truques de aceleração · Escolha de K · Misturas gaussianas · Misturas de Bernoulli — **28 perguntas**

PARTE III scikit-learn §2.1 a §2.1.1 — Gaussian mixture models

A classe `GaussianMixture` · prós e contras · `covariance_type` · seleção por BIC · o algoritmo EM na biblioteca — **14 perguntas**

PARTE IV scikit-learn §2.3 a §2.3.2 — Clustering

Panorama dos métodos · a classe `KMeans` · inércia · limitações documentadas · inicialização · MiniBatch — **16 perguntas**

PARTE V Simulado integrador

Questões que cruzam as quatro referências, no formato de arguição de banca

Questões que cruzam as quatro referências, no formato de arguição de banca.

#8212; **12 perguntas**

APÊNDICE Formulário de uma página

Todas as equações essenciais reunidas para revisão de véspera.

Como usar este guia · Notação

Convenções válidas para todas as partes seguintes.

Cada parte cobre uma referência do professor, na ordem em que ele as listou. A estrutura é sempre a mesma: primeiro o **conteúdo** da seção, com as definições e equações; depois um bloco de **perguntas e respostas** sobre aquele trecho.

As quatro referências se sobrepõem bastante — Bishop cap. 15 sozinho cobre quase tudo que as outras três discutem. Isso é uma vantagem para estudar: quando um conceito aparece pela segunda vez, ele vem com uma ênfase diferente. O guia sinaliza essas repetições em vez de escondê-las, porque é exatamente onde as bancas costumam perguntar (“o Bishop trata isso como derivação; o Murphy, como decisão de projeto; o scikit-learn, como parâmetro”).

Os três modelos deste conteúdo, em uma frase cada

- **K-Means** — particiona $\mathbb{R}^D$ em K regiões esféricas, atribuição rígida, minimiza a distorção J . Não é probabilístico.
- **Mistura de Bernoulli** — modela vetores binários $\{0, 1\}^D$ como D moedas independentes por componente, atribuição suave, maximiza $\ln p(\mathbf{X})$ via EM.
- **Mistura Gaussiana (GMM)** — modela $\mathbb{R}^D$ como combinação convexa de elipsóides, atribuição suave, maximiza $\ln p(\mathbf{X})$ via EM. Generaliza o K-Means.

§0.1 Tabela de símbolos

SÍMBOLO	SIGNIFICADO	ONDE APARECE
N, D, K	Número de amostras, de atributos (dimensões) e de componentes/clusters.	Todas as referências
$\mathbf{x}_n$	A n -ésima observação. Vetor coluna em $\mathbb{R}^D$ (Gauss) ou em $\{0, 1\}^D$ (Bernoulli).	Variável observada
$\mathbf{z}_n$	Vetor latente 1-de- K : exatamente uma coordenada vale 1, as outras 0. Diz de qual componente $\mathbf{x}_n$ veio.	Variável latente (nunca observada)
z_{nk}	A k -ésima coordenada de $\mathbf{z}_n$. Vale 1 se a amostra n pertence ao componente k .	Bishop 15.2
r_{nk}	Atribuição rígida do K-Means: 1 ou 0, sem meio-termo.	Bishop 15.1, Murphy 21.3
$\gamma(z_{nk}), \gamma_{nk}$	Responsabilidade : o posterior $p(z_{nk} = 1 \mid \mathbf{x}_n)$. Um número real em $[0, 1]$.	Bishop 15.2.1, o conceito central

π_k	Coeficiente de mistura (prior do componente k). $\sum_k \pi_k = 1$, $\pi_k \geq 0$.	Todas
μ_k	Centróide (K-Means), vetor de probabilidades por dimensão (Bernoulli) ou média do componente (GMM).	Todas
Σ_k	Matriz de covariância $D \times D$ do componente k . Controla forma, tamanho e orientação.	GMM apenas
N_k	Massa efetiva do componente k : $N_k = \sum_n \gamma(z_{nk})$. Um “número fracionário de pontos”.	Passo M do EM
θ	Conjunto de todos os parâmetros: $\{\pi_k, \mu_k, \Sigma_k\}$.	Bishop 15.4
$\mathcal{L}(q, \theta)$	A cota inferior (ELBO). Sempre $\leq \ln p(\mathbf{X} \mid \theta)$.	Bishop 15.4
$\mathbf{x}^\top$	Transposta. $\mathbf{x}^\top \mathbf{y}$ é um escalar (produto interno); $\mathbf{x} \mathbf{y}^\top$ é uma matriz.	Todas
$\ \mathbf{x}\ ^2$	Norma euclidiana ao quadrado: $\sum_i x_i^2$. É $\mathbf{x}^\top \mathbf{x}$.	K-Means

UMA ARMADILHA DE NOTAÇÃO QUE DERRUBA MUITA GENTE

μ_k quer dizer três coisas diferentes conforme o modelo, e as três são vetores de tamanho D :

- No **K-Means**, é um ponto no espaço — o centróide, a média aritmética dos pontos atribuídos.
- Na **mistura de Bernoulli**, cada coordenada $\mu_{ki} \in [0, 1]$ é a *probabilidade* de o bit i valer 1 no componente k . Não é um ponto do hipercubo: é o parâmetro de D moedas.
- No **GMM**, é a média da gaussiana — e, ao contrário dos outros dois, ela vem acompanhada de Σ_k .

Bishop, cap. 15 — Discrete Latent Variables

Bishop, C. M.; Bishop, H. *Deep Learning: Foundations and Concepts*. Springer, 2024. Capítulo 15, seções 15.1 a 15.4.5.

O capítulo constrói um arco único: começa com o K-Means como algoritmo intuitivo e sem probabilidade, revela que ele é o caso-limite de um modelo generativo (a mistura gaussiana), generaliza o procedimento de ajuste para o algoritmo EM, aplica-o também a dados binários e, por fim, prova que o EM funciona usando a decomposição da cota inferior.

15.1 K-means Clustering

O problema: dado $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ em $\mathbb{R}^D$, particionar as amostras em K grupos. A intuição que o capítulo formaliza é que um bom agrupamento tem distâncias intra-cluster pequenas comparadas às distâncias entre clusters.

Introduzem-se dois conjuntos de incógnitas. Os **protótipos** $\mu_k \in \mathbb{R}^D$, um por cluster, e as **variáveis de atribuição** $r_{nk} \in \{0, 1\}$, que formam uma codificação 1-de- K : para cada n , exatamente um r_{nk} vale 1.

A função objetivo — distorção (ou inércia)

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\mathbf{x}_n - \mu_k\|^2$$

J é a soma dos quadrados das distâncias de cada ponto ao protótipo do seu cluster.

Objetivo: encontrar $\{r_{nk}\}$ e $\{\mu_k\}$ que minimizem J .

• O algoritmo de Lloyd: descida alternada

Minimizar J simultaneamente nas duas famílias de variáveis é difícil (é NP-difícil no caso geral). A saída é uma **otimização alternada**: fixa-se uma família e minimiza-se exatamente na outra, alternadamente. Cada uma das duas subminimizações tem solução fechada.

Passo E (atribuição). Com os μ_k fixos, J é linear em r_{nk} e os termos de diferentes n são independentes. Então basta, para cada ponto, escolher o protótipo mais próximo:

$$r_{nk} = \begin{cases} 1, & \text{se } k = \arg \min_j \|\mathbf{x}_n - \mu_j\|^2 \\ 0, & \text{caso contrário.} \end{cases}$$

Passo M (atualização). Com os r_{nk} fixos, J é quadrática em μ_k . Derivando e igualando a zero:

$$2 \sum_{n=1}^N r_{nk} (\mathbf{x}_n - \mu_k) = \mathbf{0} \implies \mu_k = \frac{\sum_n r_{nk} \mathbf{x}_n}{\sum_n r_{nk}}$$

Ou seja, o novo protótipo é a **média** dos pontos atribuídos ao cluster k — daí o nome do método. O denominador $\sum_n r_{nk}$ é simplesmente quantos pontos caíram naquele cluster.

POR QUE CONVERGE

Cada um dos dois passos **reduz ou mantém** J , nunca aumenta — porque cada um resolve exatamente o seu subproblema. Como $J \geq 0$ é limitada inferiormente e o número de atribuições possíveis é finito (K^N), a sequência de valores de J é monótona não-crescente e a convergência ocorre em um número finito de passos. O que **não** se garante é que o ponto de chegada seja o mínimo global: ele é um mínimo local, e depende da inicialização.

Geometricamente, a regra de atribuição do vizinho mais próximo particiona o espaço em um **diagrama de Voronoi**: as fronteiras entre clusters são hiperplanos, perpendiculares e equidistantes aos segmentos que ligam pares de protótipos. Isso já antecipa a limitação do método — as fronteiras são sempre retas, o que só faz sentido se os grupos forem aproximadamente esféricos e de tamanho semelhante.

15.1.1 Segmentação de imagem

A aplicação que o capítulo usa para ilustrar: cada pixel de uma imagem colorida é um ponto em $\mathbb{R}^3$ (as intensidades R, G, B). Rodando K-Means sobre esses N pontos com K pequeno e substituindo cada pixel pelo μ_k do seu cluster, obtém-se uma imagem com apenas K cores.

Isso serve a dois propósitos distintos que vale não confundir:

- **Segmentação** — dividir a imagem em regiões homogêneas. Note que é uma segmentação ingênua: o K-Means sobre cores ignora completamente a posição espacial dos pixels, então regiões desconexas com a mesma cor caem no mesmo segmento.
- **Compressão com perdas** — em vez de guardar D bytes por pixel, guarda-se o índice do cluster ($\log_2 K$ bits) mais a tabela dos K protótipos. Para uma imagem de N pixels e 24 bits por pixel, passa-se de $24N$ bits para $N \log_2 K + 24K$ bits. Com $K = 8$ e uma imagem grande, isso é uma taxa próxima de 8:1.

Essa é exatamente a **quantização vetorial** que o Murphy trata em 21.3.3, com outro vocabulário: os μ_k passam a se chamar *codewords* e o conjunto deles, *codebook*.

PERGUNTAS 1–12 · K-MEANS

01 O que exatamente a função J mede, e por que ela usa a distância ao quadrado em vez da distância simples?

J mede a soma das distâncias quadradas de cada ponto ao protótipo do cluster ao qual ele foi atribuído — é a “energia” total de dispersão interna. O quadrado não é decorativo: ele é o que torna o passo M analiticamente solúvel. Derivando uma soma de quadrados em relação a μ_k obtém-se uma equação *linear*, cuja solução é a média. Se usássemos a distância euclidiana simples (norma L_2 sem quadrado), o minimizador seria a **mediana geométrica**, que não tem forma fechada e exige um procedimento iterativo próprio. O quadrado é o que compra a solução em uma linha.

02 Prove que a média minimiza J em relação a μ_k .

Com os r_{nk} fixos, isole os termos que dependem de μ_k : $J_k = \sum_n r_{nk} \|\mathbf{x}_n - \mu_k\|^2$. O gradiente em relação a μ_k é $\nabla J_k = -2 \sum_n r_{nk} (\mathbf{x}_n - \mu_k)$. Igualando a zero: $\sum_n r_{nk} \mathbf{x}_n = \mu_k \sum_n r_{nk}$, logo $\mu_k = \frac{\sum_n r_{nk} \mathbf{x}_n}{\sum_n r_{nk}}$, a média. É um *mínimo* (e não máximo ou sela) porque a segunda derivada é $2 \sum_n r_{nk} > 0$ sempre que o cluster for não vazio — a função é uma parábola com concavidade para cima em cada coordenada.

03 Por que se garante que o K-Means converge, mas não que ele encontre o melhor agrupamento?

São duas afirmações distintas. **Convergência**: cada passo minimiza exatamente o seu subproblema, então J nunca aumenta; como $J \geq 0$ e só existem K^N atribuições possíveis, a sequência estabiliza em número finito de iterações. **Otimidade**: a descida alternada só garante que não há melhora movendo *uma* família de variáveis por vez. Isso caracteriza um mínimo local, não global — existem configurações em que trocar simultaneamente vários pontos e vários centróides reduziria J , mas nenhum passo isolado consegue. Encontrar o mínimo global é NP-difícil, e por isso a prática é rodar várias vezes com inicializações diferentes e ficar com o menor J (o parâmetro `n_init` do scikit-learn).

04 O que é um diagrama de Voronoi e por que ele descreve as fronteiras do K-Means?

Dado um conjunto de pontos-semente, a célula de Voronoi de uma semente é o conjunto de todos os pontos do espaço mais próximos dela do que de qualquer outra. Como a regra de atribuição do K-Means é literalmente “pertence ao protótipo mais próximo”, a partição induzida é, por definição, o diagrama de Voronoi dos μ_k . A fronteira entre dois clusters j e k é o conjunto onde $\|\mathbf{x} - \mu_j\| = \|\mathbf{x} - \mu_k\|$, que é o hiperplano mediador do segmento $\mu_j \mu_k$. Consequência prática: **as fronteiras são sempre lineares**, o que condena o K-Means em dados com separação curva.

05 Por que a atribuição do passo E pode ser feita ponto a ponto, independentemente?

Porque, com os protótipos fixos, $J = \sum_n (\sum_k r_{nk} \|\mathbf{x}_n - \mu_k\|^2)$ é uma **soma de termos que não interagem**: a escolha de r_{nk} para a amostra n não aparece em nenhum outro termo da soma. Minimizar uma soma de parcelas independentes é minimizar cada parcela separadamente. E cada parcela é apenas “escolha um k ”, cujo mínimo é o k de menor distância. Essa separabilidade é o que faz o passo E custar $O(NKD)$ e ser trivialmente paralelizável.

06 O que acontece se um cluster ficar vazio durante a execução?

O denominador $\sum_n r_{nk}$ vira zero e a atualização $\mu_k = 0/0$ fica indefinida. Implementações reais tratam isso de alguma forma: reposicionar o protótipo órfão sobre o ponto mais distante do seu centróide atual, sobre um ponto sorteado, ou dividir o cluster de maior distorção. O scikit-learn realocaliza. Vale notar que clusters vazios são um sintoma de má inicialização — com k-means++ eles são muito mais raros, porque as sementes já nascem espalhadas.

97 Qual o custo computacional de uma iteração?

$O(NKD)$. O passo E calcula $N \times K$ distâncias, cada uma custando $O(D)$ operações. O passo M percorre os N pontos uma vez, somando vetores de tamanho D , o que é $O(ND)$ e portanto dominado pelo passo E. O custo total é $O(NKD)$ por iteração, vezes o número de iterações até a convergência. É por isso que o K-Means escala bem: é linear em cada uma das três grandezas.

98 Por que a codificação 1-de- K é usada para r_{nk} em vez de um único inteiro $c_n \in \{1, \dots, K\}$?

Nada impede guardar um inteiro na implementação — e é o que se faz na prática. A codificação 1-de- K existe pela **álgebra**: ela transforma um “escolha um caso” em uma soma ponderada, $\sum_k r_{nk}(\cdot)$, que pode ser derivada, somada e manipulada como qualquer expressão contínua. Um índice inteiro não se deriva. É a mesma razão pela qual o vetor latente $\mathbf{z}_n$ aparece codificado assim em 15.2: permite escrever $p(\mathbf{z}) = \prod_k \pi_k^{z_k}$, que colapsa exatamente em π_j quando $z_j = 1$, porque todos os outros fatores viram $\pi_k^0 = 1$.

99 O K-Means é um modelo generativo? Ele consegue calcular $p(\mathbf{x})$ ou gerar dados novos?

Não, em nenhum dos dois casos. O K-Means é um **procedimento de otimização**, não um modelo probabilístico: ele não define nenhuma distribuição sobre $\mathbf{x}$. Não há como perguntar “qual a probabilidade desta observação?”, nem como amostrar um ponto novo, nem como comparar dois valores de K por verossimilhança. Tudo o que ele entrega é uma partição e os centróides. Essa ausência é a motivação para os modelos de mistura: eles fazem o agrupamento e entregam $p(\mathbf{x})$, o que abre detecção de anomalias, geração de amostras e seleção de modelo por critérios como o BIC.

10 Na segmentação de imagem, qual é a taxa de compressão obtida e de onde ela vem?

Uma imagem RGB de N pixels ocupa $24N$ bits (8 bits por canal). Após a quantização com K cores, guarda-se: (i) para cada pixel, o índice do seu cluster, $\lceil \log_2 K \rceil$ bits; (ii) a tabela dos K protótipos, $24K$ bits. Total: $N \lceil \log_2 K \rceil + 24K$ bits. Com $K = 8$, isso é $3N + 192$ contra $24N$ — cerca de $8\times$. A compressão vem de trocar a cor exata de cada pixel pelo índice de uma cor aproximada; é **com perdas**, e o erro total introduzido é exatamente a distorção J .

11 Por que padronizar os atributos antes de rodar K-Means, e o que acontece se você não fizer isso?

Porque a distância euclidiana soma as diferenças de todas as dimensões com peso igual, mas as dimensões têm unidades arbitrárias. Se um atributo é “renda em reais” (escala de 10^4) e outro é “anos de estudo” (escala de 10^1), a diferença ao quadrado do primeiro domina a soma por seis ordens de grandeza: na prática o algoritmo agrupa *apenas* por renda, e os anos de estudo não influenciam nada. Padronizar (subtrair a média e dividir pelo desvio, o `StandardScaler`) coloca todos os atributos na mesma escala e devolve a eles voz igual. A decisão de padronizar ou não é uma escolha de modelagem, não uma formalidade: você está declarando quais atributos importam igualmente.

12 Como o K-Means se comporta em dimensão muito alta?

Mal, por causa da concentração de distâncias — um dos rostos da maldição da dimensionalidade. À medida que D cresce, as distâncias euclidianas entre pares de pontos tendem a se igualar: a razão entre a maior e a menor distância de um ponto aos demais converge para 1. Quando todo mundo está aproximadamente à mesma distância de todo mundo, “o protótipo mais próximo” deixa de ser uma informação discriminante e as atribuições ficam quase arbitrárias. A recomendação prática (que a própria documentação do scikit-learn faz) é reduzir a dimensionalidade antes — tipicamente com PCA —, o que além de mitigar o problema acelera bastante a execução.

15.2 Mixtures of Gaussians

Aqui o capítulo muda de registro: em vez de um algoritmo, define-se um **modelo generativo** — uma história de como os dados teriam sido produzidos. A mistura gaussiana escreve a densidade como uma combinação convexa de K gaussianas:

A densidade da mistura

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad \sum_{k=1}^K \pi_k = 1, \quad 0 \leq \pi_k \leq 1$$

As duas restrições sobre os π_k não são arbitrárias: elas são exatamente o que garante que $p(\mathbf{x}) \geq 0$ e que $\int p(\mathbf{x}) d\mathbf{x} = 1$, isto é, que a mistura seja de fato uma densidade.

A gaussiana multivariada que aparece em cada componente é

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\boldsymbol{\Sigma}|^{1/2}} \exp\left\{-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right\}$$

O expoente $\Delta^2 = (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})$ é a **distância de Mahalanobis** ao quadrado. Ela generaliza a distância euclidiana levando em conta a dispersão e a correlação dos dados: quando $\boldsymbol{\Sigma} = \mathbf{I}$, ela se reduz a $\|\mathbf{x} - \boldsymbol{\mu}\|^2$. As superfícies de densidade constante são elipsóides cujos eixos são os autovetores de $\boldsymbol{\Sigma}$, com semi-eixos proporcionais às raízes dos autovalores.

• A variável latente e a formulação equivalente

A construção decisiva do capítulo é reescrever a mesma densidade **introduzindo uma variável latente** $\mathbf{z}$ de K dimensões, com codificação 1-de- K . Define-se:

$$p(\mathbf{z}) = \prod_{k=1}^K \pi_k^{z_k}, \quad p(\mathbf{x} \mid \mathbf{z}) = \prod_{k=1}^K \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_k}$$

Como exatamente um z_k vale 1, cada produto colapsa em um único fator. Marginalizando a variável latente recupera-se a mistura:

$$p(\mathbf{x}) = \sum_{\mathbf{z}} p(\mathbf{z}) p(\mathbf{x} \mid \mathbf{z}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

Por que dar essa volta se o resultado é o mesmo? Porque a formulação com $\mathbf{z}$ abre duas portas que a forma marginal fecha: ela dá uma **história generativa** (sorteie o componente, depois sorteie o ponto dentro dele) e, sobretudo, ela permite perguntar pelo **posterior** de $\mathbf{z}$ dado $\mathbf{x}$ — que é o motor de todo o algoritmo EM.

15.2.1 Função de verossimilhança e responsabilidades

Responsabilidade — a quantidade central do capítulo

$$\gamma(z_k) \equiv p(z_k = 1 | \mathbf{x}) = \frac{p(z_k = 1) p(\mathbf{x} | z_k = 1)}{\sum_{j=1}^K p(z_j = 1) p(\mathbf{x} | z_j = 1)} = \frac{\pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

É o teorema de Bayes aplicado ao vetor latente: π_k é o **prior** (o quanto o componente k é comum), a densidade $\mathcal{N}(\cdot)$ é a **verossimilhança** (o quanto o componente k explica *esta* observação), e o denominador é a evidência que normaliza. O resultado é a fração da amostra $\mathbf{x}$ pela qual o componente k se responsabiliza.

Duas propriedades a memorizar: $\gamma(z_k) \in [0, 1]$ e, para toda amostra, $\sum_{k=1}^K \gamma(z_{nk}) = 1$. A responsabilidade é uma **distribuição de probabilidade sobre os componentes** — é isso que caracteriza a atribuição *suave*, em contraste com o $r_{nk} \in \{0, 1\}$ do K-Means.

Para o conjunto completo $\mathbf{X}$, supondo amostras i.i.d., a log-verossimilhança é

$$\ln p(\mathbf{X} | \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

15.2.2 Máxima verossimilhança: os dois obstáculos

Obstáculo 1 — o somatório preso dentro do logaritmo. Na expressão acima, o $\ln$ está aplicado a uma *soma*. Como $\ln(a + b) \neq \ln a + \ln b$, o logaritmo não consegue atravessar a soma e quebrar o produto das densidades em termos separados. Se houvesse um único componente, o $\ln$ cancelaria a exponencial da gaussiana e sobraria uma quadrática com solução fechada. Com a soma no meio, derivar e igualar a zero produz um sistema de equações acopladas sem solução analítica: os $\boldsymbol{\mu}_k$ ótimos dependem das responsabilidades, que dependem dos $\boldsymbol{\mu}_k$. É a razão de existir do EM.

Obstáculo 2 — singularidades. A função de verossimilhança de uma mistura gaussiana é **ilimitada superiormente**. Suponha que um componente k colapse exatamente sobre uma única observação, $\boldsymbol{\mu}_k = \mathbf{x}_n$, e que sua covariância encolha, $\boldsymbol{\Sigma}_k = \sigma_k^2 \mathbf{I}$ com $\sigma_k \rightarrow 0$. A contribuição daquele ponto é

$$\mathcal{N}(\mathbf{x}_n | \mathbf{x}_n, \sigma_k^2 \mathbf{I}) = \frac{1}{(2\pi)^{D/2} \sigma_k^D} \rightarrow +\infty$$

e portanto $\ln p(\mathbf{X}) \rightarrow +\infty$. Essa singularidade é um artefato patológico, não uma solução útil: o modelo “explicou” um ponto perfeitamente e ignorou todos os outros. Ela é uma consequência estrutural da máxima verossimilhança em misturas — em um modelo de **uma única** gaussiana ela não ocorre, porque o componente único precisa dar conta de todos os pontos ao mesmo tempo. As defesas práticas são regularizar a covariância (somar $\epsilon \mathbf{I}$, o

`reg_covar` do scikit-learn), reinicializar o componente colapsado, ou adotar uma abordagem bayesiana com prior sobre Σ .

Um terceiro detalhe: identificabilidade. Uma mistura de K componentes tem $K!$ soluções equivalentes, obtidas permutando os rótulos dos componentes. Isso não atrapalha a densidade ajustada (ela é idêntica), mas significa que “o componente 1” não tem significado estável entre execuções — importante quando se quer interpretar os componentes.

PERGUNTAS 13–24 · MISTURAS GAUSSIANAS

13 Qual a diferença entre $\mathbf{x}$ e $\mathbf{z}$, e por que $\mathbf{z}$ é chamada de latente?

$\mathbf{x}$ é **observada**: está no arquivo de dados, foi medida. $\mathbf{z}$ é **latente**: nunca foi medida e nunca estará no arquivo — ela representa o fator gerador escondido (de qual componente aquela amostra veio). Latente significa exatamente isso: existe no modelo, tem papel causal na história generativa, mas não é acessível. Todo o aparato do capítulo — responsabilidades, EM, ELBO — existe para inferir $\mathbf{z}$ a partir de $\mathbf{x}$. Se $\mathbf{z}$ fosse observada, o problema viraria classificação supervisionada e teria solução fechada trivial.

14 Por que $\sum_k \gamma(z_{nk}) = 1$ para toda amostra n ?

Porque $\gamma(z_{nk})$ é o posterior $p(z_{nk} = 1 \mid \mathbf{x}_n)$, e os eventos $\{z_{n1} = 1\}, \dots, \{z_{nK} = 1\}$ são mutuamente exclusivos e exaustivos: a amostra veio de *exatamente um* componente, e necessariamente de algum deles. Uma distribuição de probabilidade sobre um conjunto exaustivo de eventos disjuntos soma 1. Algebricamente, isso se vê direto na fórmula: o denominador é a soma de todos os numeradores possíveis, então somar em k dá $\frac{\sum_k \text{num}_k}{\sum_j \text{num}_j} = 1$. É a checagem número um ao depurar uma implementação de EM.

15 Interprete π_k e explique por que ele é diferente de $\gamma(z_{nk})$.

π_k é o **prior**: a probabilidade de uma amostra qualquer, escolhida antes de você olhar para ela, ter vindo do componente k . É uma propriedade global do modelo, um número só, igual para todas as amostras. $\gamma(z_{nk})$ é o **posterior**: a probabilidade de *aquela* amostra específica ter vindo do componente k , depois de ver o seu valor. É uma propriedade local, um número por par (amostra, componente) — uma matriz $N \times K$. A relação entre os dois é o teorema de Bayes, e a ponte é a verossimilhança $\mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$. Detalhe que fecha o círculo: no passo M, o novo π_k é a média das responsabilidades, $\pi_k = N_k/N$ — o prior é reestimado como a média dos posteriores.

16 Por que introduzir a variável latente se $p(\mathbf{x}) = \sum_k \pi_k \mathcal{N}(\cdot)$ já descreve a mesma densidade?

Porque a forma marginal esconde a estrutura que torna o problema tratável. Com $\mathbf{z}$ explícita, a verossimilhança **completa** $p(\mathbf{X}, \mathbf{Z} \mid \theta)$ tem uma propriedade que a marginal não tem: o logaritmo entra e separa tudo, porque $\ln p(\mathbf{X}, \mathbf{Z}) = \sum_n \sum_k z_{nk} \{\ln \pi_k + \ln \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\}$ — os z_{nk} ficam *fora* do logaritmo, e a expressão vira uma soma de termos independentes com solução fechada. O EM explora exatamente isso: como não conhecemos $\mathbf{Z}$, substituímos z_{nk} pelo seu valor esperado $\gamma(z_{nk})$ e maximizamos. Além disso, a formulação latente dá a história generativa e o posterior, que a marginal não oferece.

17 Explique a distância de Mahalanobis e por que ela é mais adequada que a euclidiana em um GMM.

$\Delta^2 = (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})$. A inversa da covariância age como uma mudança de métrica: ela normaliza cada direção pela dispersão dos dados naquela direção e descorrelaciona os eixos. Na prática, ela responde “a quantos desvios-padrão deste componente o ponto está?”, em vez de “a quantas unidades?”. Isso importa porque um ponto a 3 unidades de distância ao longo de uma direção onde os dados variam muito é *típico*, enquanto o mesmo deslocamento numa direção de pouca variação é *anômalo*. A euclidiana trata as duas situações igual; a Mahalanobis não. Quando $\boldsymbol{\Sigma} = \mathbf{I}$ as duas coincidem — a euclidiana é o caso particular em que se assume variância unitária e correlação zero, que é exatamente a hipótese implícita do K-Means.

18 O que cada parte da matriz de covariância $\boldsymbol{\Sigma}$ controla, geometricamente?

Em 2-D, $\boldsymbol{\Sigma} = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix}$. As entradas da diagonal, σ_1^2 e σ_2^2 , controlam a **largura** da elipse ao longo de cada eixo. A entrada fora da diagonal, via a correlação ρ , controla a **inclinação**: com $\rho = 0$ a elipse fica alinhada aos eixos; com $\rho \rightarrow \pm 1$ ela se estreita e gira para $\pm 45^\circ$. Formalmente, os autovetores de $\boldsymbol{\Sigma}$ dão as direções dos eixos principais da elipse e os autovalores dão o quadrado dos semi-eixos. Uma $\boldsymbol{\Sigma}$ precisa ser simétrica e positiva definida — todos os autovalores estritamente positivos — para que a elipse exista e a densidade seja bem definida.

19 Por que não existe solução fechada para os parâmetros de máxima verossimilhança de um GMM?

Por causa do somatório dentro do logaritmo em $\ln p(\mathbf{X}) = \sum_n \ln \{\sum_k \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\}$. Como o $\ln$ não distribui sobre a soma, ele não cancela a exponencial das gaussianas. Derivando em relação a $\boldsymbol{\mu}_k$ e igualando a zero, chega-se a $\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_n \gamma(z_{nk}) \mathbf{x}_n$ — que *parece* uma solução, mas não é: $\gamma(z_{nk})$ depende de $\boldsymbol{\mu}_k$, $\boldsymbol{\Sigma}_k$ e π_k . O sistema é implícito e acoplado. O EM transforma esse impasse em um procedimento: congele as responsabilidades, resolva para os parâmetros, recalcule as responsabilidades, repita.

20 Descreva a singularidade da verossimilhança e por que ela não aparece em uma gaussiana única.

Se um componente centra em um único ponto ($\mu_k = \mathbf{x}_n$) e sua covariância encolhe ($\sigma_k \rightarrow 0$), a densidade naquele ponto diverge como $1/\sigma_k^D \rightarrow \infty$, e com ela a log-verossimilhança total. O ótimo global da verossimilhança é, portanto, uma configuração degenerada e inútil. Em um modelo de **uma única** gaussiana isso não acontece porque o componente é obrigado a explicar *todos* os N pontos: encolher a variância sobre um deles despenca a densidade de todos os outros, e o produto total vai a zero, não a infinito. A patologia é específica de misturas, onde os demais componentes “cobrem” o resto dos dados e liberam um componente para colapsar impunemente.

21 Como se defende na prática contra as singularidades?

Três caminhas, em ordem de sofisticação. (i) **Regularizar a covariância**: somar $\epsilon \mathbf{I}$ à diagonal a cada passo M , garantindo $|\Sigma_k| > 0$ e invertibilidade. É o `reg_covar` do `scikit-learn`, ativo por padrão em 10^{-6} . (ii) **Detectar e reinicializar**: monitorar componentes cuja covariância ou massa N_k desabam e reposicioná-los aleatoriamente. (iii) **Abordagem bayesiana**: colocar um prior sobre Σ_k (tipicamente Wishart inversa) e maximizar o posterior em vez da verossimilhança — o prior penaliza covariâncias degeneradas e a singularidade desaparece. É o que faz a `BayesianGaussianMixture`.

22 O que é o problema de identificabilidade em misturas?

Permutar os rótulos dos K componentes produz um conjunto de parâmetros diferente mas uma densidade $p(\mathbf{x})$ **idêntica**. Existem, portanto, $K!$ soluções equivalentes, e o ótimo não é único como ponto no espaço de parâmetros. Isso é inofensivo se você só quer a densidade ou a partição, mas tem duas consequências reais: (i) “o componente 2” não significa a mesma coisa entre duas execuções, então interpretar componentes exige alinhá-los antes; (ii) tomar médias de parâmetros entre execuções ou entre amostras de um MCMC é sem sentido sem resolver a permutação (o “label switching”).

23 Quantos parâmetros livres tem um GMM com K componentes em D dimensões, com covariâncias completas?

Três blocos. As médias: KD . As covariâncias: cada Σ_k é simétrica $D \times D$, logo tem $D(D+1)/2$ entradas livres, totalizando $K D(D+1)/2$. Os pesos: K valores com a restrição $\sum_k \pi_k = 1$, logo $K - 1$ livres. Total: $KD + K \frac{D(D+1)}{2} + K - 1$. O termo quadrático em D é o que domina e explica por que covariâncias completas ficam caras rápido: com $D = 50$ e $K = 5$, só as covariâncias já somam 6 375 parâmetros. É esse crescimento que motiva as variantes `tied`, `diag` e `spherical`.

24 Um GMM é sempre melhor que o K-Means?

Não. Ele é mais *expressivo*, o que é diferente de melhor. O GMM ganha quando os grupos são elipsoidais, de tamanhos ou orientações diferentes, quando há sobreposição real e a incerteza da atribuição importa, ou quando se precisa de $p(\mathbf{x})$ para anomalias, geração ou seleção de modelo. O K-Means ganha quando os grupos *são* aproximadamente esféricos e separados, quando N ou D são grandes (o GMM tem muito mais parâmetros e itera mais devagar), quando não há dados suficientes para estimar K covariâncias sem degenerar, ou quando se quer um resultado simples e reproduzível. Mais parâmetros com poucos dados significa variância alta e risco de singularidade — a expressividade extra do GMM pode custar caro.

15.3 O algoritmo EM (Expectation-Maximization)

O EM resolve o impasse do “ovo e a galinha”: para calcular as responsabilidades preciso dos parâmetros; para estimar os parâmetros preciso das responsabilidades. A saída é alternar, cada passo assumindo o outro como resolvido.

O esqueleto do EM, válido para qualquer modelo latente

1. **Inicialize** os parâmetros θ^{old} .
2. **Passo E** — com θ^{old} fixo, calcule o posterior das latentes, $p(\mathbf{Z} | \mathbf{X}, \theta^{\text{old}})$, isto é, as responsabilidades.
3. **Passo M** — com as responsabilidades fixas, maximize a esperança da log-verossimilhança completa:
 $\theta^{\text{new}} = \arg \max_{\theta} Q(\theta, \theta^{\text{old}})$, com $Q(\theta, \theta^{\text{old}}) = \sum_{\mathbf{Z}} p(\mathbf{Z} | \mathbf{X}, \theta^{\text{old}}) \ln p(\mathbf{X}, \mathbf{Z} | \theta)$.
4. **Verifique** a convergência de $\ln p(\mathbf{X} | \theta)$ ou dos parâmetros; se não convergiu, volte ao passo 2.

A razão de o passo M ser fácil está em como o logaritmo se comporta na verossimilhança **completa**. Enquanto $\ln p(\mathbf{X} | \theta)$ tem uma soma presa dentro do log, a versão completa tem os z_{nk} *fora* dele:

$$\ln p(\mathbf{X}, \mathbf{Z} | \theta) = \sum_{n=1}^N \sum_{k=1}^K z_{nk} \{ \ln \pi_k + \ln p(\mathbf{x}_n | \theta_k) \}$$

Tomando a esperança sobre $\mathbf{Z}$, cada z_{nk} é substituído pelo seu valor esperado, que é precisamente $\gamma(z_{nk})$. O resultado é uma soma de termos desacoplados, cada um com máximo em forma fechada.

15.3.1 EM para misturas gaussianas

Passo E: $\gamma(z_{nk}) = \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$

Passo M: definindo a massa efetiva $N_k = \sum_{n=1}^N \gamma(z_{nk})$,

$$\boldsymbol{\mu}_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) \mathbf{x}_n$$

$$\boldsymbol{\Sigma}_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (\mathbf{x}_n - \boldsymbol{\mu}_k^{\text{new}})(\mathbf{x}_n - \boldsymbol{\mu}_k^{\text{new}})^\top \quad \pi_k^{\text{new}} = \frac{N_k}{N}$$

Leitura das três fórmulas: são exatamente as estimativas de máxima verossimilhança de uma gaussiana isolada — média amostral, covariância amostral, frequência relativa — só que **ponderadas pelas responsabilidades**. Cada ponto contribui para cada componente na proporção em que aquele componente o reivindica. Note que N_k é fracionário: é o “número efetivo de pontos” do componente k .

15.3.2 Relação com o K-means

Este é o resultado que unifica as duas primeiras seções do capítulo. Considere um GMM restrito, com covariâncias esféricas iguais e fixas, $\boldsymbol{\Sigma}_k = \epsilon \mathbf{I}$ para todo k , e pesos iguais $\pi_k = 1/K$. A responsabilidade fica

$$\gamma(z_{nk}) = \frac{\exp\{-\|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2/2\epsilon\}}{\sum_j \exp\{-\|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2/2\epsilon\}}$$

Quando $\epsilon \rightarrow 0$, o termo cujo expoente é menos negativo — o do protótipo **mais próximo** — domina todos os outros, e a razão converge para 1 para esse componente e 0 para os demais. Isto é: $\gamma(z_{nk}) \rightarrow r_{nk}$. As responsabilidades suaves endurecem, o passo E vira a atribuição do vizinho mais próximo, o passo M vira a média simples, e a esperança da log-verossimilhança completa se torna $-\frac{1}{2\epsilon} \sum_n \sum_k r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2 + \text{const.}$ Maximizar isso é **minimizar a distorção** J .

O elo

$$\boldsymbol{\Sigma}_k = \epsilon \mathbf{I}, \quad \pi_k = 1/K, \quad \epsilon \rightarrow 0 \quad \implies \quad \gamma(z_{nk}) \rightarrow r_{nk} \in \{0, 1\} \quad \text{e} \quad \text{EM} \rightarrow \text{K-Means}$$

O K-Means é o **caso-limite rígido** do EM para misturas gaussianas. Ele é rápido justamente porque descartou tudo que o GMM carrega: a forma dos componentes, os pesos e a incerteza da atribuição.

15.3.3 Misturas de distribuições de Bernoulli

Para dados **binários** $\mathbf{x} \in \{0, 1\}^D$ — pixels preto-e-branco, presença/ausência de palavras, respostas sim/não — a gaussiana é inadequada: ela põe massa de probabilidade em pontos que não existem, e a média de vetores binários não é um vetor binário. A troca é modelar cada componente como D moedas independentes.

$$p(\mathbf{x} \mid \boldsymbol{\mu}_k) = \prod_{i=1}^D \mu_{ki}^{x_i} (1 - \mu_{ki})^{1-x_i}, \quad p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\pi}) = \sum_{k=1}^K \pi_k p(\mathbf{x} \mid \boldsymbol{\mu}_k)$$

Cada $\mu_{ki} \in [0, 1]$ é a probabilidade de o bit i valer 1 no componente k . Em imagens de dígitos binarizados, o vetor $\boldsymbol{\mu}_k$ visualizado como imagem é um **protótipo em tons de cinza**: pixels quase sempre acesos naquele dígito ficam claros, pixels ambíguos ficam intermediários.

Note a hipótese embutida: dentro de um componente, os D bits são **condicionalmente independentes**. Isso é claramente falso (pixels vizinhos são correlacionados), mas a mistura recupera boa parte da correlação — a dependência entre os bits é capturada pela variável latente, não dentro de cada componente. A covariância da mistura não é diagonal, embora a de cada componente seja:

$$\text{cov}[\mathbf{x}] = \sum_{k=1}^K \pi_k \{ \Sigma_k + \mu_k \mu_k^\top \} - \mathbb{E}[\mathbf{x}] \mathbb{E}[\mathbf{x}]^\top, \quad \Sigma_k = \text{diag}(\mu_{ki}(1 - \mu_{ki}))$$

EM para a mistura de Bernoulli. Mesma estrutura, densidade trocada:

$$\gamma(z_{nk}) = \frac{\pi_k p(\mathbf{x}_n | \mu_k)}{\sum_j \pi_j p(\mathbf{x}_n | \mu_j)} \quad \mu_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) \mathbf{x}_n \quad \pi_k^{\text{new}} = \frac{N_k}{N}$$

O passo M é a **média ponderada dos vetores binários** — e o resultado, sendo média de zeros e uns, cai naturalmente em $[0, 1]$, que é exatamente o domínio válido para μ_k . Não há covariância a estimar, o que torna a mistura de Bernoulli muito mais barata e estável que o GMM, e imune ao problema de singularidade por colapso de covariância.

DOIS MODOS DE QUEBRAR O EM EM BERNOULLI, E AS DEFESAS

1. Verossimilhança nula. Se algum μ_{ki} chegar exatamente a 0 ou 1, qualquer amostra que discorde naquele bit recebe $p(\mathbf{x} | \mu_k) = 0$, e o parâmetro trava para sempre (a média ponderada não consegue mais movê-lo). **Defesa:** suavização de Laplace, $\mu_{ki} = \frac{\sum_n \gamma_{nk} x_{ni} + \alpha}{N_k + 2\alpha}$, que é o estimador MAP com prior Beta e mantém μ_{ki} estritamente dentro de $(0, 1)$.

2. Subfluxo numérico. $p(\mathbf{x} | \mu_k)$ é um produto de D fatores menores que 1. Com $D = 784$ (uma imagem 28×28), o produto fica abaixo do menor `float64` representável e colapsa para 0 — e a razão do passo E vira 0/0. **Defesa:** trabalhar em espaço logarítmico e usar o truque **Log-Sum-Exp**: $\ln \sum_k e^{a_k} = a_{\max} + \ln \sum_k e^{a_k - a_{\max}}$. Subtrair o máximo garante que o maior expoente seja $e^0 = 1$ e que nada transborde nem desapareça.

PERGUNTAS 25–38 · O ALGORITMO EM

25 Explique o EM em uma frase, e depois em quatro.

Em uma: alterne entre estimar a distribuição das variáveis latentes com os parâmetros atuais e reestimar os parâmetros como se essa distribuição fosse os dados verdadeiros.

Em quatro: (1) inicialize os parâmetros; (2) no passo E, calcule as responsabilidades $\gamma(z_{nk})$ pelo teorema de Bayes; (3) no passo M, recalcule π_k , μ_k (e Σ_k , no GMM) como estatísticas amostrais ponderadas por essas responsabilidades; (4) repita até $\ln p(\mathbf{X})$ estabilizar.

26 Por que o passo M tem solução fechada se a máxima verossimilhança direta não tem?

Porque o passo M não maximiza $\ln p(\mathbf{X} | \theta)$; ele maximiza $\mathcal{Q}(\theta, \theta^{\text{old}})$, a esperança da log-verossimilhança **completa**. Nessa expressão os z_{nk} estão fora do logaritmo, então $\ln p(\mathbf{X}, \mathbf{Z} | \theta) = \sum_n \sum_k z_{nk} \{\ln \pi_k + \ln p(\mathbf{x}_n | \theta_k)\}$ se decompõe em termos que envolvem um componente de cada vez. Cada termo é a log-verossimilhança de uma distribuição simples (uma gaussiana, uma Bernoulli), cujo máximo é conhecido em forma fechada. O somatório problemático desapareceu porque assumimos $\mathbf{Z}$ conhecido (em esperança).

27 O que é N_k e por que ele é fracionário?

$N_k = \sum_n \gamma(z_{nk})$ é o **número efetivo de pontos** do componente k : a soma das frações de cada amostra que aquele componente reivindica. É fracionário porque as atribuições são suaves — um ponto na fronteira entre dois componentes pode contribuir 0,5 para cada. No K-Means, com $r_{nk} \in \{0, 1\}$, o análogo seria uma contagem inteira. Duas propriedades úteis: $\sum_k N_k = N$ (a massa total se conserva), e $\pi_k = N_k/N$ no passo M — o peso do componente é a sua fatia da massa total.

28 As fórmulas do passo M para o GMM parecem as estimativas usuais de uma gaussiana. Qual a diferença?

São literalmente as mesmas fórmulas, com um peso $\gamma(z_{nk})$ dentro de cada soma e N_k no lugar de N no denominador. Se as responsabilidades fossem 0/1 (atribuição rígida), você recuperaria exatamente a média e a covariância amostrais dos pontos daquele cluster. A ponderação é a única novidade, e ela codifica a ideia central do EM: como não sabemos a qual componente cada ponto pertence, deixamos cada ponto contribuir para *todos* os componentes, na proporção da sua responsabilidade.

29 Por que Σ_k^{new} usa μ_k^{new} e não μ_k^{old} ?

Porque o passo M maximiza $\mathcal{Q}$ em relação a *todos* os parâmetros conjuntamente, e a solução ótima para a covariância é a dispersão em torno da **nova** média. Usar a média antiga daria uma covariância maior que a ótima (pelo teorema do eixo paralelo, a dispersão em torno de qualquer ponto que não seja a média é sempre maior). Na implementação, isso significa uma ordem obrigatória dentro do passo M: calcule N_k , depois μ_k^{new} , e só então Σ_k^{new} . Inverter a ordem é um erro silencioso — o algoritmo ainda roda e ainda converge, mas para um ponto ligeiramente errado.

30 Demonstre por que o K-Means é o caso-limite do EM para GMM.

Fixe $\Sigma_k = \epsilon \mathbf{I}$ e $\pi_k = 1/K$. Os fatores constantes de normalização e o π_k cancelam entre numerador e denominador, deixando $\gamma(z_{nk}) = \frac{\exp\{-\|\mathbf{x}_n - \mu_k\|^2/2\epsilon\}}{\sum_j \exp\{-\|\mathbf{x}_n - \mu_j\|^2/2\epsilon\}}$. Seja j^* o índice do protótipo mais próximo. Dividindo numerador e denominador por $\exp\{-\|\mathbf{x}_n - \mu_{j^*}\|^2/2\epsilon\}$, cada termo $k \neq j^*$ vira $\exp\{-(d_k^2 - d_{j^*}^2)/2\epsilon\}$ com $d_k^2 - d_{j^*}^2 > 0$, que tende a 0 quando $\epsilon \rightarrow 0$. Sobra 1 para j^* e 0 para o resto: $\gamma \rightarrow r$. Substituindo em $\mathcal{Q}$, obtém-se $-\frac{1}{2\epsilon} \sum_n \sum_k r_{nk} \|\mathbf{x}_n - \mu_k\|^2 + \text{const} = -\frac{J}{2\epsilon} + \text{const}$. Maximizar em μ é minimizar J .

31 Por que a gaussiana é uma péssima escolha para dados binários?

Três razões que se reforçam. (i) **Suporte errado**: a gaussiana espalha densidade por todo $\mathbb{R}^D$, atribuindo massa a pontos como $(0,37; -1,2)$ que não podem existir em $\{0,1\}^D$. (ii) **Média sem sentido como protótipo**: em $\{0,1\}^D$ os dados vivem nos 2^D vértices do hipercubo, e a média de vértices cai no *interior*, que não é um ponto válido do domínio. (iii) **Geometria enganosa**: a distância euclidiana entre vetores binários vira a raiz da distância de Hamming, e as noções de forma e orientação que Σ captura não têm análogo útil aqui. A mistura de Bernoulli resolve os três: o suporte é o hipercubo, o parâmetro μ_{ki} é uma probabilidade e não um ponto, e não há covariância a interpretar.

32 O que representa cada coordenada do vetor μ_k na mistura de Bernoulli?

μ_{ki} é a probabilidade de o bit i valer 1 dado que a amostra veio do componente k . Em imagens binarizadas 8×8 , μ_k tem 64 coordenadas e, visualizado como imagem, é o protótipo do dígito daquele componente em tons de cinza: $\mu_{ki} \approx 1$ onde o traço quase sempre passa, ≈ 0 no fundo, e valores intermediários onde há variação de estilo entre as amostras — que é justamente onde a incerteza mora. Diferente de um centróide de K-Means, μ_k não é um ponto dos dados: é um parâmetro que descreve D moedas.

33 A mistura de Bernoulli assume independência entre os bits. Como ela consegue então capturar correlações?

A independência é **condicional**, válida apenas *dentro* de um componente. A distribuição marginal $p(\mathbf{x}) = \sum_k \pi_k \prod_i \mu_{ki}^{x_i} (1 - \mu_{ki})^{1-x_i}$ não fatora, e sua covariância — dada pela fórmula de 15.3.3 — tem termos fora da diagonal. A intuição: se o componente “dígito 1” tem uma coluna central acesa e o componente “dígito 0” tem um anel, então observar um pixel do anel aceso aumenta a probabilidade de outros pixels do anel também estarem acesos — não porque haja dependência direta entre eles, mas porque ambos são evidência do mesmo componente. A correlação é mediada pela variável latente. É a mesma estrutura do Naive Bayes, e a razão de ele funcionar melhor do que a sua hipótese ingênua sugeriria.

34 Por que o passo M da mistura de Bernoulli é mais simples e estável que o do GMM?

Porque não há covariância. O passo M inteiro é $\mu_k = \frac{1}{N_k} \sum_n \gamma_{nk} \mathbf{x}_n$ e $\pi_k = N_k/N$: uma média ponderada e uma fração. Consequências: o custo cai de $O(NKD^2)$ para $O(NKD)$; não há matriz para inverter nem determinante para calcular, então nenhuma questão de condicionamento numérico; e não existe o modo de falha por $|\Sigma_k| \rightarrow 0$, porque não existe Σ_k . O número de parâmetros também é muito menor: $KD + K - 1$ contra $KD + KD(D + 1)/2 + K - 1$. A mistura de Bernoulli tem seus próprios modos de falha (o travamento em 0 ou 1 e o subfluxo), mas ambos têm defesas simples e baratas.

35 O que é a suavização de Laplace e qual problema ela resolve?

Substitui $\mu_{ki} = \frac{\sum_n \gamma_{nk} x_{ni}}{N_k}$ por $\mu_{ki} = \frac{\sum_n \gamma_{nk} x_{ni} + \alpha}{N_k + 2\alpha}$, tipicamente com $\alpha = 1$. Resolve o travamento em 0 ou 1: se nenhuma amostra do componente k jamais acender o bit i , o estimador sem suavização crava $\mu_{ki} = 0$, e a partir daí qualquer amostra que acenda aquele bit recebe verossimilhança exatamente zero naquele componente — o que é uma afirmação absurda de certeza a partir de evidência finita, e é irreversível, porque o numerador da média ponderada fica permanentemente zero. Formalmente, é o estimador MAP com um prior $\text{Beta}(\alpha + 1, \alpha + 1)$ sobre cada μ_{ki} , e corresponde a “fingir” ter visto α exemplos de cada resultado antes de olhar os dados. No Bishop, é o assunto de 15.4.3 (priors sobre parâmetros).

36 Explique o truque Log-Sum-Exp e por que ele é indispensável aqui.

Queremos $\ln \sum_k e^{a_k}$ sem calcular e^{a_k} diretamente, porque com a_k muito negativo (o log de um produto de centenas de probabilidades) a exponencial vira 0 em `float64`, e o log de 0 é $-\infty$. O truque é fatorar o maior termo: seja $a_{\max} = \max_k a_k$; então $\ln \sum_k e^{a_k} = a_{\max} + \ln \sum_k e^{a_k - a_{\max}}$. Agora todos os expoentes são ≤ 0 , o maior deles é exatamente 0 (dando $e^0 = 1$), e a soma está garantidamente entre 1 e K — nem transborda nem desaparece. É matematicamente uma identidade exata, não uma aproximação. Sem ele, uma mistura de Bernoulli sobre imagens de 784 pixels simplesmente não roda: $0,5^{784} \approx 10^{-236}$ já está no limite, e valores típicos são bem menores.

37 Como se inicializa o EM na prática, e por que a inicialização importa?

Importa porque o EM converge para um **máximo local**, e a superfície de verossimilhança de uma mistura é cheia deles. As estratégias comuns: (i) rodar K-Means primeiro e usar os centróides como μ_k , as covariâncias empíricas de cada cluster como Σ_k e as proporções como π_k — é o padrão do scikit-learn (`init_params='kmeans'`); (ii) sortear K pontos dos dados como médias; (iii) k-means++ para as sementes. Rodar várias inicializações e ficar com a de maior $\ln p(\mathbf{X})$ é a prática recomendada (`n_init`). Um detalhe importante: como o K-Means já é o caso-limite do EM, inicializar com ele coloca o EM num ponto razoável do espaço e reduz muito o número de iterações necessárias.

38 Quando se deve parar o EM?

O critério padrão é a estabilização da log-verossimilhança: parar quando $\ln p(\mathbf{X} | \theta^{\text{new}}) - \ln p(\mathbf{X} | \theta^{\text{old}}) < \text{tol}$, com uma tolerância relativa da ordem de 10^{-3} a 10^{-6} (o parâmetro `tol`), somada a um teto de iterações (`max_iter`). Alternativamente, monitorar a variação dos parâmetros. Uma observação prática que serve de teste de sanidade da implementação: como a teoria garante que $\ln p(\mathbf{X})$ **nunca diminui**, qualquer queda observada é bug — tipicamente uma ordem trocada no passo M, um esquecimento de renormalizar as responsabilidades, ou perda de precisão numérica.

15.4 Evidence Lower Bound (ELBO)

As seções anteriores *usaram* o EM. Esta explica **por que ele funciona** — por que a log-verossimilhança nunca diminui, e o que exatamente cada passo está fazendo

geometricamente.

A construção parte de uma identidade válida para **qualquer** distribuição $q(\mathbf{Z})$ sobre as variáveis latentes:

A decomposição fundamental

$$\ln p(\mathbf{X} | \boldsymbol{\theta}) = \mathcal{L}(q, \boldsymbol{\theta}) + \text{KL}(q \| p)$$
$$\mathcal{L}(q, \boldsymbol{\theta}) = \sum_{\mathbf{Z}} q(\mathbf{Z}) \ln \left\{ \frac{p(\mathbf{X}, \mathbf{Z} | \boldsymbol{\theta})}{q(\mathbf{Z})} \right\}, \quad \text{KL}(q \| p) = - \sum_{\mathbf{Z}} q(\mathbf{Z}) \ln \left\{ \frac{p(\mathbf{Z} | \mathbf{X}, \boldsymbol{\theta})}{q(\mathbf{Z})} \right\}$$

Como a divergência de Kullback-Leibler satisfaz $\text{KL}(q \| p) \geq 0$, com igualdade se e somente se $q(\mathbf{Z}) = p(\mathbf{Z} | \mathbf{X}, \boldsymbol{\theta})$, segue imediatamente que

$$\mathcal{L}(q, \boldsymbol{\theta}) \leq \ln p(\mathbf{X} | \boldsymbol{\theta})$$

ou seja, $\mathcal{L}$ é uma **cota inferior** da log-verossimilhança — daí o nome *evidence lower bound* (a “evidência” é $p(\mathbf{X})$).

15.4.1 O EM revisitado: os dois passos como dois movimentos

Com a decomposição em mãos, os dois passos do EM ganham uma leitura geométrica limpa.

Passo E. Mantendo $\boldsymbol{\theta}^{\text{old}}$ fixo, maximiza-se $\mathcal{L}(q, \boldsymbol{\theta}^{\text{old}})$ em relação a q . Como a soma $\mathcal{L} + \text{KL}$ é constante (não depende de q), maximizar $\mathcal{L}$ é **zerar a KL**. Isso ocorre exatamente quando $q(\mathbf{Z}) = p(\mathbf{Z} | \mathbf{X}, \boldsymbol{\theta}^{\text{old}})$ — o posterior, isto é, as responsabilidades. Nesse ponto a cota **encosta** na log-verossimilhança: $\mathcal{L}(q, \boldsymbol{\theta}^{\text{old}}) = \ln p(\mathbf{X} | \boldsymbol{\theta}^{\text{old}})$.

Passo M. Mantendo q fixo, maximiza-se $\mathcal{L}(q, \boldsymbol{\theta})$ em relação a $\boldsymbol{\theta}$. Substituindo q pelo posterior encontrado, $\mathcal{L}(q, \boldsymbol{\theta}) = \mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}}) + \text{const}$, onde a constante é a entropia de q — que não depende de $\boldsymbol{\theta}$. Maximizar a cota é, portanto, maximizar $\mathcal{Q}$: exatamente o passo M que já conhecíamos.

POR QUE $\ln p(\mathbf{X})$ NUNCA DIMINUI

Ao fim do passo E, $\mathcal{L} = \ln p(\mathbf{X} | \boldsymbol{\theta}^{\text{old}})$ e $\text{KL} = 0$. O passo M aumenta $\mathcal{L}$ (ou a mantém). Mas ao mudar $\boldsymbol{\theta}$, o posterior $p(\mathbf{Z} | \mathbf{X}, \boldsymbol{\theta}^{\text{new}})$ muda também, enquanto q ficou congelado — logo a KL volta a ser ≥ 0 . Somando: $\ln p(\mathbf{X} | \boldsymbol{\theta}^{\text{new}}) = \mathcal{L}(q, \boldsymbol{\theta}^{\text{new}}) + \text{KL} \geq \mathcal{L}(q, \boldsymbol{\theta}^{\text{new}}) \geq \mathcal{L}(q, \boldsymbol{\theta}^{\text{old}}) = \ln p(\mathbf{X} | \boldsymbol{\theta}^{\text{old}})$. A log-verossimilhança sobe pelo menos tanto quanto a cota subiu, e ainda ganha o incremento da KL. **Monotonicidade garantida.**

15.4.2 Dados independentes e identicamente distribuídos

Quando as N observações são i.i.d., o posterior conjunto sobre $\mathbf{Z}$ fatora por amostra: $p(\mathbf{Z} | \mathbf{X}, \boldsymbol{\theta}) = \prod_n p(\mathbf{z}_n | \mathbf{x}_n, \boldsymbol{\theta})$. É isso que torna o passo E viável: em vez de lidar com uma distribuição sobre K^N configurações conjuntas, calcula-se N distribuições independentes de K valores cada — a matriz $N \times K$ de responsabilidades. Sem a hipótese i.i.d., o passo E exigiria inferência sobre um objeto exponencialmente grande.

15.4.3 Priors sobre os parâmetros

Se em vez de máxima verossimilhança quisermos o **MAP** (maximum a posteriori), com um prior $p(\theta)$, a decomposição vira $\ln p(\theta | \mathbf{X}) = \mathcal{L}(q, \theta) + \text{KL}(q||p) + \ln p(\theta) + \text{const.}$ O passo E fica idêntico; o passo M passa a maximizar $\mathcal{Q}(\theta, \theta^{\text{old}}) + \ln p(\theta)$. Duas aplicações diretas do que já vimos: a suavização de Laplace na mistura de Bernoulli é o passo M com prior Beta, e o `reg_covar` do GMM é uma versão prática do passo M com prior sobre a covariância. O prior **elimina as singularidades**, porque penaliza covariâncias degeneradas e probabilidades exatamente 0 ou 1.

15.4.4 EM generalizado (GEM)

Nada na prova da monotonicidade exige que o passo M encontre o máximo *global* de $\mathcal{Q}$. Basta que ele **aumente** $\mathcal{Q}$. Isso libera o EM generalizado: quando o passo M não tem solução fechada, pode-se dar alguns passos de subida de gradiente, ou maximizar em um subconjunto dos parâmetros por vez (o ECM, *expectation conditional maximization*). A garantia de que $\ln p(\mathbf{X})$ não diminui permanece intacta. Simetricamente, existe uma versão generalizada do passo E, em que se maximiza $\mathcal{L}$ apenas dentro de uma família restrita de q — é o ponto de partida da **inferência variacional**.

15.4.5 EM sequencial

Em bases grandes, o passo E percorre todas as N amostras antes de qualquer atualização de parâmetro. O EM sequencial (ou *online*) processa uma amostra (ou um lote) por vez: calcula a responsabilidade daquela amostra e atualiza os parâmetros incrementalmente, usando estatísticas suficientes acumuladas. Converge mais rápido em tempo de relógio para N grande e permite aprendizado em fluxo, sem manter os dados em memória. É a mesma ideia que leva do K-Means clássico ao `MiniBatchKMeans` do scikit-learn.

PERGUNTAS 39–46 · ELBO E CONVERGÊNCIA

39 Enuncie a decomposição da log-verossimilhança e diga o que cada termo significa.

$\ln p(\mathbf{X} | \theta) = \mathcal{L}(q, \theta) + \text{KL}(q||p)$, válida para qualquer $q(\mathbf{Z})$. O termo $\mathcal{L}$ é a **cota inferior** (ELBO): o quanto conseguimos garantir sobre a log-verossimilhança usando a aproximação q . O termo $\text{KL}(q||p)$ mede o **desencontro** entre a aproximação q e o posterior verdadeiro $p(\mathbf{Z} | \mathbf{X}, \theta)$; é sempre ≥ 0 e vale 0 apenas quando as duas coincidem. A soma é fixa para um θ dado, então melhorar q significa transferir valor da KL para a cota.

40 Por que $\text{KL}(q||p) \geq 0$, e por que isso é o que torna $\mathcal{L}$ uma cota inferior?

A não-negatividade da KL segue da desigualdade de Jensen aplicada ao logaritmo, que é côncavo: $-\sum_{\mathbf{Z}} q \ln \frac{p}{q} \geq -\ln \sum_{\mathbf{Z}} q \frac{p}{q} = -\ln \sum_{\mathbf{Z}} p = -\ln 1 = 0$. A igualdade em Jensen exige que o argumento seja constante, o que aqui significa $p(\mathbf{Z} | \mathbf{X})/q(\mathbf{Z})$ constante — isto é, $q = p$. Como $\ln p(\mathbf{X}) = \mathcal{L} + \text{KL}$ e a KL nunca é negativa, subtraí-la só pode reduzir: $\mathcal{L} \leq \ln p(\mathbf{X})$.

41 O que o passo E faz, em termos de $\mathcal{L}$ e KL?

Com θ congelado, $\ln p(\mathbf{X} | \theta)$ é um número fixo, e portanto $\mathcal{L}$ e KL são complementares: o que uma ganha, a outra perde. O passo E escolhe o q que **zera a KL**, o que é o mesmo que levar $\mathcal{L}$ ao seu valor máximo possível para aquele θ . Esse q ótimo é o posterior exato, $p(\mathbf{Z} | \mathbf{X}, \theta^{\text{old}})$ — as responsabilidades. Geometricamente: a cota, que estava abaixo da curva de log-verossimilhança, **sobe até tocá-la** no ponto θ^{old} .

42 E o passo M?

Congela q e move θ para maximizar $\mathcal{L}(q, \theta)$. Como $\mathcal{L}(q, \theta) = \mathcal{Q}(\theta, \theta^{\text{old}}) + H[q]$, e a entropia $H[q]$ não depende de θ , maximizar a cota é maximizar $\mathcal{Q}$. Geometricamente: a cota, que agora toca a curva em θ^{old} , é **escalada até o seu próprio máximo**. Como ela está sempre abaixo da log-verossimilhança e a tocava no ponto de partida, subir na cota obriga a log-verossimilhança a subir pelo menos o mesmo tanto.

43 Prove que o EM nunca diminui a log-verossimilhança.

Encadeie quatro relações. (1) Após o passo E: $\mathcal{L}(q, \theta^{\text{old}}) = \ln p(\mathbf{X} | \theta^{\text{old}})$, pois a KL foi zerada. (2) O passo M garante $\mathcal{L}(q, \theta^{\text{new}}) \geq \mathcal{L}(q, \theta^{\text{old}})$, por definição de maximização. (3) Para qualquer θ , vale $\ln p(\mathbf{X} | \theta) = \mathcal{L}(q, \theta) + \text{KL} \geq \mathcal{L}(q, \theta)$. Aplicando em θ^{new} e juntando: $\ln p(\mathbf{X} | \theta^{\text{new}}) \geq \mathcal{L}(q, \theta^{\text{new}}) \geq \mathcal{L}(q, \theta^{\text{old}}) = \ln p(\mathbf{X} | \theta^{\text{old}})$. O ganho real é ainda maior que o ganho na cota, porque a KL reabre e soma. **Cuidado:** monotonicidade não é convergência ao ótimo global — a sequência sobe, mas pode estabilizar em um máximo local ou ponto de sela.

44 O que é o EM generalizado e por que ele ainda funciona?

É o EM em que o passo M apenas **aumenta** $\mathcal{Q}$ em vez de maximizá-la. Continua funcionando porque a prova da monotonicidade só usa $\mathcal{L}(q, \theta^{\text{new}}) \geq \mathcal{L}(q, \theta^{\text{old}})$ — a desigualdade não exige o máximo. Isso é útil quando o passo M não tem solução fechada: dá-se alguns passos de gradiente, ou maximiza-se um bloco de parâmetros por vez (ECM). Analogamente existe um passo E generalizado, em que q é otimizado dentro de uma família restrita e a KL não chega a zero — a cota fica frouxa, mas ainda é cota. Esse é o passo de partida da inferência variacional, e é o que a `BayesianGaussianMixture` faz.

45 Como um prior sobre os parâmetros se encaixa no EM, e o que ele resolve?

Trocando o alvo de $\ln p(\mathbf{X} | \theta)$ para $\ln p(\theta | \mathbf{X}) \propto \ln p(\mathbf{X} | \theta) + \ln p(\theta)$. O passo E não muda em nada; o passo M passa a maximizar $\mathcal{Q}(\theta, \theta^{\text{old}}) + \ln p(\theta)$. O ganho decisivo é que o prior **elimina as singularidades**: um prior que atribui densidade zero a covariâncias degeneradas (ou a probabilidades exatamente 0 e 1) torna essas configurações inatingíveis, e a função objetivo deixa de ser ilimitada. As duas defesas práticas que já vimos são exatamente isso: a suavização de Laplace é o passo M com prior Beta, e o `reg_covar` aproxima o passo M com prior sobre Σ .

46 Qual a vantagem do EM sequencial sobre o EM em lote?

No EM em lote, nenhum parâmetro se move antes de percorrer todas as N amostras — com N na casa dos milhões, cada iteração é cara e o progresso é lento em tempo de relógio. O EM sequencial atualiza os parâmetros a cada amostra ou lote pequeno, usando estatísticas suficientes acumuladas, e por isso faz muito mais progresso por passagem nos dados. Vantagens adicionais: não exige manter o conjunto inteiro em memória e permite aprendizado em fluxo contínuo. O custo é que as garantias de monotonicidade estrita se enfraquecem — o objetivo passa a oscilar em torno da trajetória de subida, como em qualquer método estocástico. É a mesma troca do `MiniBatchKMeans` : convergência mais rápida, resultado ligeiramente pior.

Murphy, 21.3 a 21.4.2

Murphy, K. P. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022. Da seção 21.3 (*K means clustering*) até 21.4.2 (*Mixtures of Bernoullis*), inclusive.

O Murphy cobre os mesmos modelos do Bishop, mas com ênfase diferente: menos derivação, mais **decisão de projeto**. É aqui que aparecem as perguntas práticas — como escolher K , como inicializar, quais variantes do algoritmo existem e quando usar cada uma. Trate esta parte como o complemento operacional da Parte I.

21.3 K means clustering

A apresentação do algoritmo é a mesma do Bishop 15.1 — atribuição ao centróide mais próximo, recálculo pela média, iterar — com uma diferença de enquadramento útil: o Murphy trata o K-means desde o início como um **método de quantização**, isto é, como um dispositivo que substitui cada ponto pelo seu representante mais próximo. Esse enquadramento torna natural tudo o que vem depois.

A função objetivo é chamada de **distorção**:

$$J(\mathbf{M}, \mathbf{Z}) = \sum_{n=1}^N \|\mathbf{x}_n - \boldsymbol{\mu}_{z_n}\|^2$$

onde $z_n \in \{1, \dots, K\}$ é o índice do cluster da amostra n (o Murphy usa índice inteiro em vez da codificação 1-de- K) e $\mathbf{M}$ é a matriz dos centróides.

• Quantização vetorial

O ponto de vista da compressão. Um conjunto de N vetores reais de dimensão D , com C bits por número, ocupa NDC bits. Substituindo cada vetor pelo índice do seu centróide, guarda-se:

- $N\lceil \log_2 K \rceil$ bits para os índices (o *código*);
- KDC bits para a tabela de centróides (o *codebook*).

Com $K \ll N$, o primeiro termo domina e a taxa de compressão é aproximadamente $DC/\lceil \log_2 K \rceil$. O erro introduzido é a distorção J — há uma troca explícita entre taxa e distorção, que é o assunto clássico da teoria da informação. O caso $K = N$ dá distorção zero e nenhuma compressão; o caso $K = 1$ dá compressão máxima e distorção máxima.

• O algoritmo k-means++

A inicialização aleatória uniforme é ruim: se duas sementes caem no mesmo grupo real, o algoritmo tende a dividir aquele grupo e fundir dois outros, e a descida alternada não consegue mais se recuperar. O **k-means++** (Arthur & Vassilvitskii, 2007) sorteia as sementes de forma a espalhá-las:

1. Escolha o primeiro centro uniformemente entre os dados.
2. Para cada ponto $\mathbf{x}$, calcule $D(\mathbf{x})$, a distância ao centro **já escolhido** mais próximo.
3. Sorteie o próximo centro entre os dados, com probabilidade $p(\mathbf{x}) = \frac{D(\mathbf{x})^2}{\sum_n D(\mathbf{x}_n)^2}$.
4. Repita 2-3 até ter K centros; então rode o Lloyd normalmente.

O peso D^2 (e não D , nem uma escolha determinística do ponto mais distante) é o que dá a garantia teórica: o k-means++ é $O(\log K)$ -competitivo com o ótimo global **em esperança**, antes mesmo de rodar uma iteração do Lloyd. O quadrado favorece fortemente pontos distantes sem tornar a escolha determinística — o que evitaria a captura por *outliers*.

• O algoritmo K-medoids

Variante em que os representantes são obrigatoriamente **pontos do conjunto de dados** (medoides), e não médias calculadas. O passo de atualização deixa de ser “tire a média” e passa a ser “escolha, entre os pontos do cluster, aquele que minimiza a soma das dissimilaridades aos demais”. O algoritmo clássico é o PAM (*Partitioning Around Medoids*).

Três vantagens sobre o K-means: (i) funciona com **qualquer** medida de dissimilaridade, inclusive não-métricas e matrizes de distância pré-calculadas — não é preciso que a média sequer exista; (ii) é bem mais **robusto a outliers**, porque um ponto extremo não consegue arrastar um medoide da mesma forma que arrasta uma média; (iii) o representante é um exemplo real, o que ajuda na interpretação. A desvantagem é o custo: o passo de atualização é $O(N_k^2)$ por cluster, contra $O(N_k D)$ do K-means.

• Truques de aceleração

O gargalo é o passo E, com suas NK distâncias por iteração. As técnicas de aceleração exploram a **desigualdade triangular** para provar que certas distâncias não precisam ser calculadas: se sabemos que $d(\mathbf{x}, \mu_j) \geq 2d(\mathbf{x}, \mu_k)$, então μ_j não pode ser o mais próximo e a distância nem precisa ser avaliada. Mantendo cotas superiores e inferiores por ponto e atualizando-as pelo quanto cada centróide se moveu, o algoritmo de **Elkan** elimina a maioria dos cálculos em dados de baixa dimensão. Outras rotas: estruturas espaciais (*k-d trees*) e amostragem por mini-lotes.

• Escolhendo o número de clusters K

O ponto em que o Murphy é mais explícito que o Bishop. O problema de fundo: J **decrece monotonicamente** com K — com $K = N$ ela é exatamente zero. Portanto **não se pode escolher K minimizando J** . São apresentadas três saídas.

MÉTODO	COMO FUNCIONA	LIMITAÇÃO
Cotovelo da distorção	Plote $J(K)$ contra K e procure o “joelho”: o ponto a partir do qual acrescentar clusters deixa de compensar. A intuição é que, abaixo do K correto, cada cluster novo divide um grupo real e reduz muito J ; acima, ele apenas fatia grupos já homogêneos.	Subjetivo e frequentemente não há cotovelo visível . É uma heurística, não um critério.

Verossimilhança marginal	Só disponível em modelos probabilísticos (GMM, mistura de Bernoulli): compare $p(\mathcal{D} K)$ entre valores de K . Aqui a curva tem máximo genuíno , porque a navalha de Occam bayesiana penaliza automaticamente modelos complexos demais — eles espalham a massa de probabilidade e explicam mal qualquer conjunto específico. Aproximações práticas: BIC e AIC.	Exige um modelo generativo. O K-means, não sendo probabilístico, está fora.
Coefficiente de silhueta	Mede simultaneamente coesão e separação, sem precisar de rótulos. Para cada ponto i : $a(i)$ é a distância média aos pontos do próprio cluster; $b(i)$ é a menor distância média a algum outro cluster. Escolhe-se o K que maximiza a silhueta média.	Custo $O(N^2)$, e tende a favorecer clusters convexos e bem separados — herda a hipótese geométrica que está tentando avaliar.

Coefficiente de silhueta

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1]$$

$s(i) \approx 1$: o ponto está muito mais perto do seu cluster do que do vizinho mais próximo — bem agrupado. $s(i) \approx 0$: está na fronteira, a atribuição é ambígua. $s(i) < 0$: está em média **mais perto de outro cluster** que do próprio — provavelmente mal atribuído.

21.4 Clustering usando modelos de mistura

A virada conceitual: em vez de um algoritmo que particiona, um **modelo probabilístico** que gera. A vantagem prática que o Murphy destaca é que isso transforma o clustering em um problema de ajuste de modelo, com todas as ferramentas que vêm junto — verossimilhança, seleção de modelo, quantificação de incerteza, e a possibilidade de amostrar dados novos.

A distinção operacional que se ganha é entre **hard clustering** (cada ponto recebe um rótulo) e **soft clustering** (cada ponto recebe uma distribuição sobre rótulos). O hard clustering se recupera do soft por $\hat{z}_n = \arg \max_k \gamma_{nk}$, mas a informação descartada nessa conversão — a confiança da atribuição — é frequentemente a mais valiosa.

21.4.1 Misturas gaussianas

Mesmo modelo da Parte I: $p(\mathbf{x}) = \sum_k \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$, ajustado por EM. O Murphy enfatiza o resultado de que **o K-means é um caso especial do EM** para GMM, sob três restrições: covariâncias esféricas e iguais ($\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$), pesos iguais ($\pi_k = 1/K$) e o limite $\sigma^2 \rightarrow 0$, que endurece as responsabilidades. Nessa leitura, o K-means aparece como uma variante de “**hard EM**”: em vez de usar o posterior completo no passo E, usa-se apenas o seu modo.

21.4.2 Misturas de Bernoulli

Para dados binários, cada componente é um produto de D Bernoullis independentes:

$$p(\mathbf{x} \mid z = k, \boldsymbol{\theta}) = \prod_{d=1}^D \text{Ber}(x_d \mid \mu_{dk}) = \prod_{d=1}^D \mu_{dk}^{x_d} (1 - \mu_{dk})^{1-x_d}$$

O modelo é também conhecido como *latent class analysis* na literatura de ciências sociais. O ajuste por EM é o da Parte I. O Murphy destaca a aplicação em imagens binarizadas (MNIST) e o fato de que os componentes aprendidos tendem a capturar **estilos de escrita** e não apenas classes de dígito — rodar com $K = 10$ sobre os dez dígitos raramente produz um componente por dígito, e sim uma mistura de variantes.

PERGUNTAS 47–74 · MURPHY

47 Por que não se pode escolher K minimizando a distorção J ?

Porque J é monotonicamente **não-crescente** em K : qualquer solução com K clusters pode ser transformada numa solução com $K + 1$ dividindo um cluster, o que só pode reduzir a distorção. No extremo $K = N$, cada ponto vira seu próprio centróide e $J = 0$. Minimizar J sobre K , portanto, sempre responde “ $K = N$ ” — uma resposta correta e inútil. O problema é que J mede apenas ajuste, sem nenhuma penalização por complexidade. As saídas são adicionar essa penalização (BIC, verossimilhança marginal) ou usar um critério de natureza diferente, que meça separação além de coesão (silhueta).

48 Explique o método do cotovelo e por que ele é frágil.

Plota-se $J(K)$ contra K e procura-se o ponto de inflexão onde a curva “dobra”. O raciocínio: enquanto K for menor que o número real de grupos, cada cluster adicional separa dois grupos genuinamente distintos e derruba J bastante; a partir do K correto, clusters adicionais apenas subdividem grupos já homogêneos, e o ganho vira marginal. A fragilidade tem dois lados: primeiro, em dados reais frequentemente **não existe** um cotovelo visível — a curva é suave e a escolha vira arbitrária; segundo, quando existe, sua localização é subjetiva e diferentes pessoas leem valores diferentes no mesmo gráfico. É uma heurística visual, não um critério, e por isso deve ser cruzada com a silhueta ou com o BIC.

49 Defina o coeficiente de silhueta e interprete seus valores.

Para cada ponto i : $a(i)$ é a distância média dele aos outros pontos do **seu próprio** cluster (coesão) e $b(i)$ é a menor, entre os outros clusters, da distância média dele aos pontos daquele cluster (separação ao vizinho mais próximo). Então $s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$, que fica em $[-1, 1]$. Perto de $+1$: ponto bem agrupado, muito mais próximo dos seus que dos outros. Perto de 0 : ponto na fronteira, a atribuição poderia ir para qualquer lado. **Negativo**: em média o ponto está mais perto de outro cluster que do seu — sinal de má atribuição. A silhueta média sobre todos os pontos serve como score global para comparar valores de K .

50 Por que a silhueta consegue ter um máximo em K se a distorção não consegue?

Porque ela mede **duas** coisas em tensão, não uma. Aumentar K tende a reduzir $a(i)$ — clusters menores são mais coesos —, o que empurra a silhueta para cima; mas também reduz $b(i)$, porque os clusters ficam mais próximos uns dos outros, o que a empurra para baixo. Existe um K em que essas duas forças se equilibram, e é ele que maximiza a silhueta. A distorção só enxerga a primeira força (coesão), então nada a impede de melhorar indefinidamente. O princípio geral: um critério de seleção de modelo precisa de um termo que *piora* quando o modelo fica complexo demais.

51 O que é a quantização vetorial e qual a sua relação com o K-means?

É a compressão com perdas de vetores, substituindo cada um pelo índice do seu representante mais próximo num dicionário de K *codewords*. O K-means é o algoritmo que aprende esse dicionário: os centróides são as codewords, o codebook é a matriz de centróides, e a distorção J é exatamente o erro de reconstrução total introduzido pela compressão. É a mesma coisa vista de dois ângulos — o clustering pergunta “quais são os grupos?”, a quantização pergunta “quais são os melhores K representantes?”, e a resposta ótima é a mesma.

52 Calcule a taxa de compressão da quantização vetorial.

Original: N vetores de D números com C bits cada, total NDC bits. Comprimido: $N\lceil\log_2 K\rceil$ bits de índices mais KDC bits de codebook. A razão é $\frac{NDC}{N\lceil\log_2 K\rceil + KDC}$, que para $K \ll N$ se aproxima de $\frac{DC}{\lceil\log_2 K\rceil}$. Exemplo concreto: imagem RGB ($D = 3$, $C = 8$, logo 24 bits/pixel) com $K = 64$ cores dá $24/6 = 4\times$ de compressão. Note que a taxa **não depende de N** no limite — depende só de quantos bits você gasta por símbolo, o que é a intuição central da teoria de taxa-distorção.

53 Descreva o k-means++ e explique por que o peso é $D(\mathbf{x})^2$.

Primeiro centro uniforme entre os dados; cada centro seguinte sorteado com probabilidade proporcional ao quadrado da distância ao centro já escolhido mais próximo, $p(\mathbf{x}) = D(\mathbf{x})^2 / \sum_n D(\mathbf{x}_n)^2$; repete-se até K centros. O expoente 2 equilibra duas pressões opostas. Um peso proporcional a D (sem quadrado) espalha pouco — regiões densas próximas a centros existentes ainda ganham massa demais no sorteio. Já escolher **deterministicamente** o ponto mais distante (o limite de expoente infinito) é frágil: um único outlier isolado seria escolhido sempre, desperdiçando um centro. O quadrado favorece fortemente pontos distantes mantendo a escolha aleatória, e é precisamente esse expoente que faz a demonstração de $O(\log K)$ -competitividade funcionar — não é uma constante ajustada empiricamente.

54 O que significa dizer que o k-means++ é “ $O(\log K)$ -competitivo”?

Que o valor esperado da distorção obtida **apenas pela sementeira**, antes de rodar qualquer iteração do Lloyd, é no máximo $O(\log K)$ vezes a distorção do agrupamento ótimo global: $\mathbb{E}[J_{++}] \leq 8(\ln K + 2) J_{\text{ótimo}}$. A esperança é sobre a aleatoriedade do sorteio. É uma garantia notável porque o K-means em geral não tem nenhuma — a inicialização uniforme pode ficar arbitrariamente longe do ótimo. Duas ressalvas: é uma cota *em esperança*, não uma garantia para uma execução específica, e ela vale para a inicialização, então rodar o Lloyd depois só melhora.

55 O que é o K-medoids e quando ele é preferível ao K-means?

É a variante em que cada representante é obrigatoriamente um **ponto do conjunto de dados** (o medoide), escolhido como aquele que minimiza a soma das dissimilaridades aos demais membros do cluster. Prefira-o em três situações: (i) quando a **média não existe ou não faz sentido** — dados categóricos, sequências, grafos, ou qualquer domínio em que só se tem uma matriz de dissimilaridades; (ii) quando há **outliers**, porque um ponto extremo puxa uma média mas não consegue puxar um medoide (a mesma diferença entre média e mediana); (iii) quando o representante precisa ser um **exemplo real** e interpretável — “o cliente típico deste segmento é este cliente” comunica melhor que um vetor de médias. O preço é o custo: o passo de atualização é $O(N_k^2)$ por cluster.

56 Como a desigualdade triangular acelera o K-means?

Ela permite **provar** que certas distâncias são irrelevantes sem calculá-las. O caso mais simples: se $d(\mu_j, \mu_k) \geq 2d(\mathbf{x}, \mu_k)$, então pela desigualdade triangular $d(\mathbf{x}, \mu_j) \geq d(\mu_j, \mu_k) - d(\mathbf{x}, \mu_k) \geq d(\mathbf{x}, \mu_k)$, logo j não pode ser o centro mais próximo e a distância nem precisa ser avaliada. O algoritmo de Elkan generaliza isso mantendo, para cada ponto, uma cota superior da distância ao centro atribuído e cotas inferiores para os demais, e atualizando essas cotas pelo *quanto cada centróide se moveu* a cada iteração — o que exige apenas K^2 distâncias entre centróides por iteração, em vez de NK distâncias ponto-centro. O ganho é grande em baixa dimensão e evapora em alta, onde as cotas ficam frouxas demais para eliminar candidatos.

57 Qual a diferença entre hard clustering e soft clustering?

No **hard**, cada ponto recebe exatamente um rótulo: a saída é um vetor de N inteiros. É o que o K-means produz. No **soft**, cada ponto recebe uma distribuição de probabilidade sobre os K componentes: a saída é uma matriz $N \times K$ com linhas que somam 1. É o que as misturas produzem, via responsabilidades. A conversão $\text{soft} \rightarrow \text{hard}$ é $\hat{z}_n = \arg \max_k \gamma_{nk}$, mas ela **descarta a confiança**: um ponto com $\gamma = (0,98; 0,02)$ e outro com $(0,51; 0,49)$ recebem o mesmo rótulo, embora o segundo seja essencialmente uma moeda ao ar. Em aplicações onde o custo de errar é assimétrico, essa informação é a mais valiosa que o modelo produz.

58 Por que a verossimilhança marginal tem um máximo genuíno em K , ao contrário de J ?

Pela **navalha de Occam bayesiana**. A verossimilhança marginal $p(\mathcal{D} | K) = \int p(\mathcal{D} | \theta, K) p(\theta | K) d\theta$ é uma média sobre *todos* os valores possíveis dos parâmetros, não o valor no melhor deles. Um modelo com muitos parâmetros consegue explicar bem *muitos* conjuntos de dados diferentes, e como a probabilidade total precisa somar 1 sobre todos os conjuntos possíveis, ele necessariamente espalha sua massa — atribuindo menos probabilidade a *qualquer* conjunto específico, inclusive o observado. A penalização por complexidade é automática, não precisa ser adicionada à mão. É por isso que a curva sobe, atinge o máximo no K adequado e depois **desce**. A distorção J , ao contrário, avalia o ajuste no melhor θ , e por isso só pode melhorar com mais parâmetros.

59 Em que sentido preciso o K-means é um caso especial do EM?

Sob três restrições simultâneas sobre o GMM: covariâncias esféricas e compartilhadas $\Sigma_k = \sigma^2 \mathbf{I}$, pesos uniformes $\pi_k = 1/K$, e o limite $\sigma^2 \rightarrow 0$. As duas primeiras removem os parâmetros de forma e de peso; a terceira endurece o passo E, fazendo $\gamma_{nk} \rightarrow r_{nk} \in \{0, 1\}$. Com isso, o passo E vira a atribuição ao centróide mais próximo, o passo M vira a média simples, e maximizar a esperança da log-verossimilhança completa equivale a minimizar J . Uma leitura alternativa e igualmente correta: o K-means é o **hard EM**, aquele que usa o *modo* do posterior em vez do posterior completo no passo E.

60 Que vantagens práticas um modelo de mistura oferece sobre o K-means?

Cinco, todas decorrentes de ele ser probabilístico. (1) **Densidade**: fornece $p(\mathbf{x})$, o que habilita detecção de anomalias por limiar de densidade. (2) **Incerteza**: as responsabilidades dizem quão confiável é cada atribuição. (3) **Geração**: pode-se amostrar dados sintéticos do modelo ajustado. (4) **Seleção de modelo**: pode-se comparar valores de K por verossimilhança, BIC ou AIC, sem depender de heurísticas visuais. (5) **Flexibilidade geométrica**: com covariâncias completas, os clusters podem ser elipsoidais, de tamanhos e orientações diferentes, e as fronteiras deixam de ser hiperplanos. O custo é mais parâmetros, mais iterações, e a exposição a singularidades.

61 Ao rodar uma mistura de Bernoulli com $K = 10$ sobre os dez dígitos do MNIST, você espera um componente por dígito?

Não, e essa expectativa é o erro conceitual mais comum aqui. O modelo é **não supervisionado**: ele não conhece os rótulos e não tem nenhum incentivo para alinhar componentes com classes semânticas. O que ele maximiza é a verossimilhança, e a estrutura que maximiza verossimilhança é a de **estilos visuais**, não a de categorias. Na prática, é comum ver dois componentes para o dígito 1 (o traço vertical simples e o com serifa), o 4 e o 9 fundidos por serem visualmente próximos, e assim por diante. Se você quer um componente por classe, precisa de supervisão ou de K maior que 10 seguido de um mapeamento componente→classe.

62 Quando você usaria mistura de Bernoulli em vez de GMM?

Sempre que os dados forem **binários** por natureza: pixels preto-e-branco, presença/ausência de palavras num documento (*bag of words* binarizado), respostas sim/não de um questionário, cliques, indicadores de sintomas, dados genômicos de presença de mutação. O critério é o domínio da variável, não o tamanho ou a dimensão. Usar GMM em dados binários é um erro de modelagem: a gaussiana atribui densidade a pontos que não podem existir, e a covariância descreve uma geometria que não tem análogo no hipercubo. Ganhos adicionais da Bernoulli: muito menos parâmetros ($KD + K - 1$ contra $KD + KD(D + 1)/2 + K - 1$), passo M mais barato, e imunidade ao colapso de covariância.

63 O que a inicialização por k-means++ garante que a aleatória uniforme não garante?

Uma **cota de aproximação**. Com inicialização uniforme não existe garantia alguma: é possível construir conjuntos em que a distorção final fica arbitrariamente pior que a ótima, e o modo de falha típico — duas sementes caindo no mesmo grupo real — é comum na prática e irrecuperável pela descida alternada. O k-means++ garante $\mathbb{E}[J] \leq 8(\ln K + 2) J_{\text{ótimo}}$ logo após a sementeira. Vale registrar o que ele **não** garante: não garante o ótimo global, não garante nada para uma execução isolada (a cota é em esperança), e não dispensa `n_init` — apenas reduz bastante a variância entre execuções.

64 Por que rodar o K-means várias vezes com sementes diferentes?

Porque ele converge para um mínimo **local**, e qual mínimo depende inteiramente da inicialização. Rodando n vezes e ficando com a execução de menor J , você amostra n mínimos locais e fica com o melhor — o que reduz muito a chance de aceitar uma solução ruim. É o que o parâmetro `n_init` do scikit-learn faz automaticamente. É importante notar que J é um critério **legítimo** para essa comparação, ao contrário do que ocorre na escolha de K : aqui todas as execuções têm o mesmo K e portanto a mesma complexidade, então comparar ajuste puro é justo.

65 Qual o efeito de outliers sobre o K-means, e como mitigá-lo?

Grande, porque a média não é robusta e a distorção usa a distância **ao quadrado** — um ponto a 10 unidades contribui 100 vezes mais que um a 1 unidade. Um único outlier distante pode deslocar sensivelmente um centróide, e o k-means++ chega a *aumentar* a chance de desperdiçar um centro sobre ele, já que sorteia proporcionalmente a D^2 . Mitigações: remover outliers na preparação dos dados; usar K-medoids, que é muito mais robusto; usar um GMM, em que um componente de covariância larga pode absorver os pontos dispersos sem distorcer os demais; ou aumentar K e aceitar que alguns clusters minúsculos sejam “lixeyras” de outliers.

66 Silhueta e BIC podem discordar sobre o melhor K . O que fazer?

Entender que eles perguntam coisas diferentes e decidir qual pergunta importa. A **silhueta** é geométrica e não paramétrica: mede quão separados e compactos os grupos estão, e favorece partições convexas e bem apartadas. O **BIC** é estatístico: mede quão bem um modelo generativo explica os dados, penalizado por complexidade, e pode preferir mais componentes se eles genuinamente melhoram a densidade — inclusive vários componentes descrevendo um único grupo alongado. Discordância típica: em dados anisotrópicos, a silhueta prefere K pequeno (os grupos alongados não são bem separados por métrica euclidiana) e o BIC prefere K maior (várias gaussianas modelam bem a forma alongada). Se o objetivo é **interpretar grupos**, siga a silhueta; se é **estimar densidade**, siga o BIC. E inspecione visualmente antes de decidir.

67 O que J mede que a silhueta não mede, e vice-versa?

J mede apenas **coesão**: o quanto os pontos estão próximos do seu próprio centróide. Ela ignora completamente onde estão os outros clusters — dois clusters vizinhos e sobrepostos podem ter J tão baixo quanto dois bem separados, se a dispersão interna for a mesma. A silhueta mede coesão e **separação**, via o termo $b(i)$, e por isso enxerga a sobreposição que J não vê. Em compensação, J é $O(NKD)$ e a silhueta é $O(N^2D)$, o que a torna proibitiva em bases grandes (na prática, calcula-se sobre uma amostra). Na prática usam-se as duas: J para comparar execuções com o mesmo K , silhueta para comparar valores de K .

68 Qual o custo computacional do EM para GMM por iteração, comparado ao K-means?

O K-means custa $O(NKD)$. O EM para GMM com covariâncias completas custa $O(NKD^2)$ por iteração: cada avaliação da densidade exige a forma quadrática $(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})$, que é $O(D^2)$, e há NK delas. Some a isso o custo de inverter (ou fatorar por Cholesky) K matrizes $D \times D$ a cada passo M, que é $O(KD^3)$. Com `covariance_type='diag'` o custo por avaliação cai para $O(D)$ e o EM volta a ser comparável ao K-means. Além do custo por iteração, o EM costuma precisar de mais iterações, porque tem mais parâmetros para acomodar.

69 O que acontece com o K-means quando os clusters têm tamanhos ou densidades muito diferentes?

Ele fatia o grupo grande e funde os pequenos. A razão é que J soma distâncias quadradas **sem normalizar por cluster**: um grupo volumoso contribui muito para a distorção total, e o algoritmo reduz J mais eficientemente dividindo-o do que respeitando a estrutura real. As fronteiras de Voronoi, sendo equidistantes entre centróides, também não têm como refletir que um grupo é mais disperso que outro. Um GMM lida melhor com isso por dois mecanismos independentes: os π_k permitem componentes com massas diferentes, e os $\boldsymbol{\Sigma}_k$ permitem dispersões diferentes.

70 Como você validaria um resultado de clustering sem rótulos verdadeiros?

Combinando evidências, já que não há gabarito. (i) **Índices internos**: silhueta, Calinski-Harabasz, Davies-Bouldin. (ii) **Estabilidade**: reamostre (bootstrap) ou perturbe os dados e verifique se a partição se mantém — agrupamentos genuínos são estáveis, artefatos não. (iii) **Critérios probabilísticos**, se o modelo for generativo: verossimilhança em dados de validação, BIC, AIC. (iv) **Inspeção visual** após redução de dimensionalidade (PCA, t-SNE, UMAP), lembrando que essas projeções podem criar ou destruir aparência de estrutura. (v) **Validação externa qualitativa**: os grupos fazem sentido para quem conhece o domínio? Essa última costuma ser a mais decisiva na prática, e é a que nenhum índice substitui.

71 Por que é comum rodar PCA antes de agrupar?

Três razões. (i) **Concentração de distâncias**: em alta dimensão as distâncias euclidianas se homogeneízam, e a noção de “mais próximo” perde poder discriminante; projetar em poucas componentes restaura o contraste. (ii) **Ruído**: as componentes de baixa variância frequentemente carregam mais ruído que sinal, e descartá-las melhora o agrupamento. (iii) **Custo**: o K-means é $O(NKD)$ e o GMM com covariância completa é $O(NKD^2)$ — reduzir D de 784 para 50 muda a ordem de grandeza do tempo de execução, e no caso do GMM também torna a estimação das covariâncias viável. A ressalva importante: o PCA maximiza variância, não separabilidade de grupos, então é possível (embora incomum) que ele descarte justamente a direção que separava os clusters.

72 Um cluster com silhueta média negativa. O que isso indica?

Que, em média, seus pontos estão mais próximos de outro cluster do que do próprio — o que é um diagnóstico forte de que aquele cluster não corresponde a estrutura real. Causas possíveis: K alto demais, e o algoritmo teve de inventar um grupo fatiando outro; um cluster que é apenas a interseção sobreposta de dois grupos vizinhos; ou a hipótese geométrica errada (grupos alongados cortados transversalmente pelas fronteiras de Voronoi produzem exatamente esse sintoma). A ação recomendada é olhar o **gráfico de silhueta por cluster**, não só a média global: um único cluster ruim pode ser mascarado por vários bons na média, e é a distribuição que revela o problema.

73 Em que situação o K-means supera o GMM na prática?

Quando as hipóteses do K-means são aproximadamente verdadeiras e os recursos são escassos. Concretamente: grupos aproximadamente esféricos e de tamanho semelhante; N muito grande, onde o custo $O(NKD)$ contra $O(NKD^2)$ decide; D alto com poucos dados por componente, situação em que estimar K covariâncias $D \times D$ é inviável e o GMM degenera; necessidade de resultado rápido e reproduzível; ou quando só se quer uma partição e nenhuma das capacidades probabilísticas. Um princípio geral que vale além deste conteúdo: o modelo mais expressivo só ganha se houver dados suficientes para estimar seus parâmetros extras — caso contrário a variância adicional supera o ganho de viés.

74 Como se determina K quando os métodos disponíveis discordam e não há conhecimento de domínio?

Aceite que K raramente tem uma resposta única e correta, e trate a escolha como uma decisão, não uma medição. O procedimento defensável: rode várias métricas (cotovelo, silhueta, BIC, AIC) sobre uma faixa de K ; procure a **faixa** em que várias delas são simultaneamente razoáveis, em vez de um valor exato; teste a estabilidade da partição por reamostragem dentro dessa faixa; e escolha o menor K dentro dela, pelo princípio da parcimônia. Documente o critério usado. Se a aplicação a jusante tolerar, entregar duas granularidades (por exemplo $K = 4$ e $K = 8$) costuma ser mais honesto e mais útil do que fingir precisão que os dados não sustentam.

scikit-learn §2.1 a §2.1.1 — Gaussian mixture models

scikit-learn User Guide, Gaussian mixture models. Seção 2.1 e 2.1.1 (Gaussian Mixture), inclusive.

Esta referência é curta e prática. Ela não deriva nada: assume a teoria das Partes I e II e responde “como isso vira código, e quais botões existem”. Duas coisas aqui costumam cair em prova: a lista de prós e contras que a própria documentação enuncia, e o significado dos quatro valores de `covariance_type`.

2.1 O módulo `sklearn.mixture`

O pacote implementa modelos de mistura gaussiana em duas classes principais. A `GaussianMixture` ajusta por **máxima verossimilhança via EM**. A `BayesianGaussianMixture` faz inferência variacional com priores, e tem a propriedade de conseguir *desligar* componentes desnecessários (empurrando seus pesos para perto de zero), o que a torna menos sensível à escolha de K .

Além do ajuste, o módulo oferece o cálculo de **elipsóides de confiança** para visualização e o critério **BIC** para avaliar quantos componentes usar.

2.1.1 A classe `GaussianMixture`

Os métodos que importam, e o que cada um devolve:

MÉTODO	O QUE FAZ / DEVOLVE
<code>fit(X)</code>	Ajusta o modelo por EM. Por padrão inicializa com K-Means (<code>init_params='kmeans'</code>).
<code>predict(X)</code>	Rótulo rígido de cada amostra: $\arg \max_k \gamma_{nk}$. Vetor de N inteiros.
<code>predict_proba(X)</code>	As responsabilidades γ_{nk} . Matriz $N \times K$, linhas somando 1. É a saída que o K-Means não tem.
<code>score_samples(X)</code>	A log-densidade $\ln p(\mathbf{x}_n)$ de cada amostra. Base para detecção de anomalias.
<code>score(X)</code>	A log-verossimilhança média por amostra.
<code>sample(n)</code>	Gera amostras novas do modelo ajustado — o modelo é generativo.
<code>bic(X)</code> / <code>aic(X)</code>	Critérios de seleção de modelo. Menor é melhor em ambos.

Prós e contras, como a documentação enuncia

Vantagens. *Velocidade* — é o algoritmo mais rápido para aprender modelos de mistura. *Neutralidade* — por maximizar apenas a verossimilhança, não envies as médias para zero nem força os clusters a terem tamanhos ou formatos particulares que poderiam não corresponder aos dados.

Desvantagens. *Singularidades* — quando há poucos pontos por componente, estimar as covariâncias fica mal condicionado e o algoritmo diverge para soluções de verossimilhança infinita, a menos que se regularize artificialmente. *Número de componentes* — o algoritmo sempre usará todos os componentes que tiver disponíveis, então determinar K exige dados de validação ou critérios de teoria da informação.

- **O parâmetro `covariance_type`**

A decisão de modelagem mais consequente da classe. Ela controla quantos graus de liberdade cada componente tem para descrever sua forma:

VALOR	GEOMETRIA	PARÂMETROS DE COVARIÂNCIA	QUANDO USAR
'full'	Cada componente tem sua própria elipse, com orientação livre.	$K \cdot D(D+1)/2$	Máxima flexibilidade. Exige muitos dados por componente.
'tied'	Todos os componentes compartilham a mesma matriz de covariância: mesma forma e orientação, posições diferentes.	$D(D+1)/2$	Quando os grupos têm forma parecida mas centros distintos. Bom meio-termo.
'diag'	Elipses alinhadas aos eixos: largura livre por dimensão, sem rotação.	$K \cdot D$	Alta dimensão. É o padrão prático quando D é grande.
'spherical'	Círculos (hiperesferas): uma única variância por componente.	K	O mais restrito. Próximo do K-Means, mas ainda probabilístico.

A progressão `full` → `tied` → `diag` → `spherical` é uma escada de viés-variância: menos parâmetros significa menos flexibilidade (mais viés) e estimativas mais estáveis (menos variância). Com poucos dados, um modelo restrito frequentemente supera um flexível.

- **Selecionando o número de componentes com o BIC**

$$\text{BIC} = -2 \ln \hat{L} + p \ln N \qquad \text{AIC} = -2 \ln \hat{L} + 2p$$

onde $\hat{L}$ é a verossimilhança maximizada, p o número de parâmetros livres e N o número de amostras. **Menor é melhor** nos dois. O primeiro termo premia o ajuste; o segundo penaliza a complexidade. A diferença entre eles: o BIC penaliza mais forte (pois $\ln N > 2$ assim que $N > 7$) e tende a escolher modelos mais parcimoniosos; o AIC tende a escolher modelos maiores.

Na prática, faz-se uma varredura dupla — sobre K e sobre `covariance_type` — e escolhe-se o par de menor BIC. Contando os parâmetros livres de um GMM: $p = \underbrace{(K-1)}_{\text{pesos}} + \underbrace{KD}_{\text{médias}} + \underbrace{p_{\Sigma}}_{\text{ver tabela}}$.

• O algoritmo de estimação: EM

A documentação descreve o EM como “o algoritmo mais bem estabelecido para modelos de mistura” e resume-o exatamente como a Parte I: passo E calcula as probabilidades de pertencimento a partir dos parâmetros atuais; passo M maximiza a verossimilhança esperada; itera-se com garantia de que a verossimilhança nunca diminui. Os parâmetros que controlam esse processo são `max_iter`, `tol`, `n_init`, `init_params` e `reg_covar`.

PERGUNTAS 75–88 · SCIKIT-LEARN: MISTURA GAUSSIANA

75 Qual a diferença entre `predict` e `predict_proba`, e quando a segunda importa?

`predict` devolve o rótulo rígido de cada amostra, $\arg \max_k \gamma_{nk}$: um vetor de N inteiros. `predict_proba` devolve a matriz $N \times K$ das responsabilidades, com linhas somando 1. A segunda importa sempre que a **confiança** da atribuição for relevante: para sinalizar casos ambíguos para revisão humana, para ponderar decisões a jusante pelo grau de pertencimento, para detectar regiões de sobreposição entre grupos, ou para tomar decisões com custos assimétricos. Um ponto com (0,99; 0,01) e outro com (0,52; 0,48) recebem o mesmo rótulo em `predict`, e são situações completamente diferentes.

76 Explique os quatro valores de `covariance_type` e o efeito de cada um.

`'full'`: cada componente tem sua matriz completa — elipses de forma e orientação livres, $KD(D+1)/2$ parâmetros. `'tied'`: uma única matriz compartilhada por todos — todos os componentes têm a mesma forma e orientação, só mudam de lugar, $D(D+1)/2$ parâmetros. `'diag'`: uma matriz diagonal por componente — elipses alinhadas aos eixos, sem rotação, KD parâmetros. `'spherical'`: uma variância escalar por componente — círculos, K parâmetros. Quanto menos parâmetros, mais restrita a geometria, mais estável a estimação e menor o risco de singularidade — ao custo de não conseguir descrever formas que os dados de fato têm.

77 Você tem $D = 100$ dimensões e apenas 500 amostras. Qual `covariance_type` escolher e por quê?

`'diag'` ou `'spherical'`. Com `'full'` e, digamos, $K = 5$, seriam $5 \times \frac{100 \times 101}{2} = 25\,250$ parâmetros de covariância para 500 amostras — cinquenta parâmetros por amostra. As matrizes estimadas seriam singulares ou quase, o EM divergiria ou dependeria inteiramente do `reg_covar`, e o resultado teria variância enorme. Com `'diag'` caem para $5 \times 100 = 500$; com `'spherical'`, para 5. A regra prática é ter pelo menos algumas dezenas de amostras por parâmetro estimado. Considere também reduzir D com PCA antes — frequentemente é a melhor jogada, e permite voltar a usar `'full'` no espaço reduzido.

78 O que faz `reg_covar` e por que ele vem ativo por padrão?

Soma uma constante positiva pequena à diagonal de cada matriz de covariância a cada passo M: $\Sigma_k \leftarrow \Sigma_k + \epsilon \mathbf{I}$, com $\epsilon = 10^{-6}$ por padrão. Isso garante que Σ_k permaneça estritamente positiva definida — logo invertível, com determinante estritamente positivo. Vem ativo por padrão porque a singularidade não é um caso raro: é o comportamento *esperado* da máxima verossimilhança em misturas, e sem regularização qualquer ajuste com poucos pontos por componente tende a divergir. É a materialização prática do que Bishop 15.4.3 apresenta como prior sobre os parâmetros.

79 Interprete a afirmação da documentação de que o algoritmo é “agnóstico”.

Significa que, por maximizar apenas a verossimilhança, ele não impõe estruturas que os dados talvez não tenham: não puxa as médias para a origem (como fariam métodos com regularização L_2 sobre μ) e não força os clusters a terem tamanhos iguais ou formatos específicos. Compare com o K-Means, que *impõe* geometria esférica e fronteiras equidistantes por construção. O lado B dessa neutralidade é a segunda desvantagem listada na mesma documentação: sem nenhuma pressão estrutural, o modelo usa todos os componentes que tiver e vai até a singularidade se deixarem — a ausência de viés é exatamente o que exige `reg_covar` e um critério externo para K .

80 Por que “o algoritmo sempre usará todos os componentes que tiver acesso” é um problema?

Porque significa que o modelo nunca lhe dirá que K está grande demais. Acrescentar componentes sempre aumenta (ou mantém) a verossimilhança, então não existe sinal interno de excesso: com K alto, o EM simplesmente fatia grupos reais em subgrupos artificiais e reporta uma verossimilhança melhor. A consequência prática é que K **precisa** ser decidido por um critério externo — BIC, AIC, verossimilhança em dados de validação separados, ou conhecimento de domínio. A alternativa oferecida pela própria biblioteca é a `BayesianGaussianMixture`, que com priores apropriados empurra os pesos de componentes supérfluos para perto de zero e efetivamente os desliga.

81 Escreva a fórmula do BIC e explique cada termo.

$\text{BIC} = -2 \ln \hat{L} + p \ln N$. O termo $-2 \ln \hat{L}$ mede o **desajuste**: quanto melhor o modelo explica os dados, maior $\hat{L}$ e menor esse termo. O termo $p \ln N$ é a **penalização por complexidade**: p é o número de parâmetros livres e $\ln N$ é o peso da penalização, que cresce com o tamanho da amostra. Menor BIC é melhor. A presença de $\ln N$ é o que diferencia o BIC do AIC (que usa $2p$): com muitos dados, o BIC penaliza a complexidade muito mais duramente e tende a selecionar modelos menores. A intuição por trás disso é que o BIC aproxima $-2 \ln p(\mathcal{D} | K)$, a verossimilhança marginal do Murphy 21.3.

82 Conte os parâmetros livres de um GMM com $K = 3$, $D = 2$, para cada `covariance_type`.

Base comum: pesos $K - 1 = 2$; médias $KD = 6$. Total base: 8. Somando as covariâncias: **full** = $K \frac{D(D+1)}{2} = 3 \times 3 = 9$, total **17**. **tied** = $\frac{D(D+1)}{2} = 3$, total **11**. **diag** = $KD = 6$, total **14**. **spherical** = $K = 3$, total **11**. Note que em $D = 2$ as opções `tied` e `spherical` empatam em contagem, mas descrevem geometrias bem diferentes (uma elipse comum inclinada versus três círculos de raios distintos) — a contagem de parâmetros não é o único critério.

83 Como usar um GMM para detectar anomalias?

Ajuste o modelo em dados considerados normais e use `score_samples(X)`, que devolve $\ln p(\mathbf{x}_n)$. Pontos em regiões de baixa densidade têm log-densidade baixa. Defina um limiar τ e sinalize $\ln p(\mathbf{x}) < \tau$ como anomalia. O limiar pode vir de um percentil dos escores de treino (por exemplo, o percentil 1, aceitando 1% de falsos positivos) ou ser calibrado num conjunto de validação com anomalias conhecidas. Três cuidados: o modelo precisa ter sido ajustado em dados limpos, ou ele aprende as anomalias como um componente legítimo; a densidade depende da escala das variáveis, então padronize antes; e em alta dimensão as estimativas de densidade ficam pouco confiáveis. Essa é uma capacidade que o K-Means simplesmente não tem — a distância ao centróide mais próximo é um substituto pobre, porque ignora a forma e a dispersão do cluster.

84 Por que `init_params='kmeans'` é o padrão?

Porque o EM converge para um máximo local e a inicialização decide qual. O K-Means é barato, é ele próprio o caso-limite do EM para GMM, e entrega em poucos segundos uma configuração já razoável: centróides bem posicionados, dos quais se derivam covariâncias empíricas e pesos por cluster. Partir daí reduz muito o número de iterações do EM e a chance de cair num máximo local ruim, comparado a sortear médias aleatoriamente. É um exemplo elegante de como a relação teórica entre os dois algoritmos (Bishop 15.3.2) tem consequência prática direta: usa-se o caso-limite rígido como ponto de partida para o modelo completo.

85 Quando preferir `BayesianGaussianMixture` à `GaussianMixture`?

Quando você não sabe K e não quer (ou não pode) fazer uma varredura com BIC, ou quando há poucos dados por componente e a singularidade é uma ameaça real. Ela usa inferência variacional com priores sobre os parâmetros, o que traz dois efeitos: componentes desnecessários têm seus pesos empurrados para perto de zero e efetivamente desligados — bastando definir um K generoso como teto —, e os priores regularizam as covariâncias naturalmente, eliminando as singularidades sem precisar do truque do `reg_covar`. Os custos são mais hiperparâmetros a definir (a concentração do prior de Dirichlet, em especial) e um ajuste mais lento.

86 Qual a diferença entre `score` e `score_samples` ?

`score_samples(X)` devolve um vetor de tamanho N : a log-densidade $\ln p(\mathbf{x}_n)$ de cada amostra individualmente. `score(X)` devolve um único escalar: a **média** dessas log-densidades, isto é, $\frac{1}{N} \sum_n \ln p(\mathbf{x}_n)$. Use `score_samples` para decisões por amostra, como detecção de anomalias; use `score` para avaliar o modelo como um todo, por exemplo comparando o desempenho em treino e validação. Atenção a um detalhe que confunde: como `score` é a média e não a soma, ao reconstruir a log-verossimilhança total para calcular o BIC manualmente é preciso multiplicar por N .

87 O BIC pode ser usado para comparar modelos com `covariance_type` diferentes?

Sim, e é exatamente para isso que ele serve. O BIC é comparável entre quaisquer modelos ajustados **ao mesmo conjunto de dados**, porque ele já contabiliza o número de parâmetros de cada um. Isso permite a varredura dupla que a documentação ilustra: para cada K e cada um dos quatro tipos de covariância, calcule o BIC e escolha o par de menor valor. É comum o resultado ser não óbvio — por exemplo, `tied` com $K = 5$ vencer `full` com $K = 3$, porque compartilhar a covariância libera parâmetros para modelar mais componentes. O que **não** se pode fazer é comparar BICs calculados sobre conjuntos de dados diferentes, ou com números de amostras diferentes.

88 Você ajusta um GMM e recebe um aviso de que o modelo não convergiu. O que investigar?

Na ordem: (1) `max_iter` baixo demais para o problema — aumente e veja se converge. (2) `tol` apertado demais para a escala dos seus dados. (3) K grande demais, com componentes disputando os mesmos pontos e as responsabilidades oscilando sem estabilizar. (4) Dados não padronizados, com variáveis de escalas muito diferentes, tornando as covariâncias mal condicionadas. (5) Poucos pontos por componente, empurrando o ajuste para a singularidade — aumentar `reg_covar` ou restringir o `covariance_type` costuma resolver. (6) Inicialização ruim — aumente `n_init`. A checagem mais informativa é registrar $\ln p(\mathbf{X})$ por iteração: se ela sobe e estabiliza, é só falta de iterações; se oscila, é instabilidade real; se dispara para valores muito altos, é singularidade.

scikit-learn §2.3 a §2.3.2 — Clustering

scikit-learn User Guide, Clustering. Seção 2.3, 2.3.1 (Overview of clustering methods) e 2.3.2 (K-means), inclusive.

Duas coisas aqui que não aparecem em nenhuma das outras três referências: o **panorama comparativo** dos métodos de clustering — que situa o K-Means dentro de um campo muito maior — e as **limitações do K-Means enunciadas explicitamente** pela documentação, que são material de prova frequente.

2.3 / 2.3.1 Panorama dos métodos de clustering

A documentação abre com uma observação que vale internalizar: todo algoritmo de clustering vem em duas formas — uma classe que implementa `fit` para aprender os clusters dos dados de treino, e uma função que, dados os dados, devolve um vetor de rótulos inteiros. E cada método traz consigo uma **hipótese geométrica implícita** sobre o que constitui um cluster; escolher o método é escolher essa hipótese.

MÉTODO	PARÂMETROS	ESCALABILIDADE	CASO DE USO / GEOMETRIA
K-Means	número de clusters	Muito grande N , K médio (com MiniBatch)	Uso geral, clusters uniformes, poucos, geometria plana. Distâncias entre pontos.
Affinity propagation	damping, preferência de amostra	Não escalável (N pequeno)	Muitos clusters, tamanhos desiguais, geometria não plana. Distância de grafo.
Mean-shift	bandwidth	Não escalável	Muitos clusters, tamanhos desiguais, geometria não plana. Distâncias entre pontos.
Spectral clustering	número de clusters	N médio, K pequeno	Poucos clusters, tamanhos parelhos, geometria não plana. Distância de grafo.
Ward (hierárquico)	número de clusters / limiar de distância	N e K grandes	Muitos clusters, restrições de conectividade. Distâncias entre pontos.
Agglomerative	nº de clusters, tipo de ligação, distância	N e K grandes	Muitos clusters, conectividade, distâncias não euclidianas.
DBSCAN	tamanho da vizinhança	N muito grande, K médio	Geometria não plana, tamanhos desiguais, remove outliers . Distâncias entre pontos mais próximos.

HDBSCAN	tamanho mín. de cluster e de vizinhança	N grande, K médio	Como DBSCAN, mas com densidades variáveis e hierarquia.
OPTICS	tamanho mínimo de cluster	N muito grande, K grande	Geometria não plana, densidades variáveis, remove outliers.
Misturas gaussianas	muitos	Não escalável	Estimação de densidade , geometria plana. Distâncias de Mahalanobis.
BIRCH	fator de ramificação, limiar	N e K grandes	Conjuntos grandes, remoção de outliers, redução de dados.
Bisecting K-Means	número de clusters	N e K muito grandes	Uso geral, geometria plana, estrutura hierárquica, clusters mais uniformes que o K-Means.

A coluna decisiva é a última. “Geometria plana” significa que o método assume que os clusters são blobs convexos separáveis por fronteiras retas; “não plana” significa que ele consegue capturar formas arbitrárias — anéis, espirais, variedades curvas. O K-Means e o GMM estão do lado plano; DBSCAN e spectral clustering, do lado não plano.

2.3.2 K-means

A documentação descreve o K-Means como um algoritmo que agrupa dividindo as amostras em K grupos de **variância igual**, minimizando a **inércia** (ou soma de quadrados intra-cluster):

$$\sum_{i=0}^n \min_{\mu_j \in C} (\|\mathbf{x}_i - \mu_j\|^2)$$

É a mesma J das Partes I e II, escrita com o mínimo explícito em vez das variáveis de atribuição. O algoritmo requer que o número de clusters seja especificado e escala bem para N grande.

As limitações que a documentação enuncia — material de prova

1. **A inércia assume clusters convexos e isotrópicos.** Ela responde mal a conjuntos alongados ou a variedades de forma irregular, porque a única geometria que sabe descrever é a esférica.
2. **A inércia não é uma métrica normalizada.** Sabemos apenas que valores menores são melhores e que zero é o ótimo — não há escala absoluta que permita dizer se um valor é “bom”.
3. **Em espaços de alta dimensão, as distâncias euclidianas se inflam** (a maldição da dimensionalidade). Rodar uma redução de dimensionalidade como o PCA antes do K-Means pode aliviar o problema e acelerar a execução.
4. **A convergência é garantida, mas para um mínimo local**, possivelmente distante do global — daí a necessidade de múltiplas inicializações.

• Inicialização e execuções múltiplas

O parâmetro `init='k-means++'` (padrão) inicializa os centróides distantes entre si, o que produz resultados comprovadamente melhores que a inicialização aleatória. O parâmetro `n_init` controla quantas execuções independentes são feitas, ficando com a de menor inércia.

2.3.2.1 Paralelismo de baixo nível

A implementação usa paralelismo via OpenMP no código Cython, com os dados divididos em blocos para o cálculo das distâncias — o que também melhora o uso de cache. O número de *threads* é controlado pela variável de ambiente `OMP_NUM_THREADS`. Vale distinguir esse paralelismo, que ocorre *dentro* de uma execução, do paralelismo entre as `n_init` execuções.

2.3.2.2 Mini Batch K-Means

A `MiniBatchKMeans` é uma variante que usa **mini-lotes** — subconjuntos dos dados sorteados a cada iteração — para reduzir drasticamente o tempo de convergência. A cada iteração, sorteia-se um lote, atribuem-se suas amostras aos centróides mais próximos e atualizam-se os centróides por uma média corrente, com taxa de aprendizado que decresce conforme o número de amostras já atribuídas àquele centróide. O resultado é geralmente **ligeiramente pior** que o do K-Means padrão em termos de inércia, mas obtido em uma fração do tempo — a troca que faz sentido quando N é muito grande.

89 O que significa a coluna “geometria” na tabela comparativa, e por que ela é a mais importante?

Ela declara a **hipótese implícita** de cada método sobre o que é um cluster e qual métrica usa para decidir. “Geometria plana” significa que o método assume blobs convexos separáveis por fronteiras retas — é o caso do K-Means (distâncias entre pontos) e do GMM (distâncias de Mahalanobis). “Não plana” significa capacidade de capturar formas arbitrárias, como anéis concêntricos ou espirais — é o caso do DBSCAN e do spectral clustering, que trabalham com conectividade ou distância de grafo. É a coluna mais importante porque nenhuma quantidade de ajuste de hiperparâmetros conserta uma hipótese geométrica errada: rodar K-Means em dois anéis concêntricos vai falhar para qualquer K , qualquer inicialização e qualquer `n_init`.

90 Escreva a inércia como a documentação a define e relacione com a distorção J .

$\sum_{i=0}^n \min_{\mu_j \in C} (\|\mathbf{x}_i - \mu_j\|^2)$: para cada amostra, a distância quadrada ao centróide mais próximo; soma-se sobre todas. É **exatamente** a mesma quantidade que $J = \sum_n \sum_k r_{nk} \|\mathbf{x}_n - \mu_k\|^2$ do Bishop, apenas escrita de outra forma: como r_{nk} vale 1 só para o centróide mais próximo, a soma interna em k seleciona precisamente o mínimo. A documentação usa “inércia”, o Bishop usa “distorção”, o Murphy também usa “distorção” — três nomes para a mesma soma de quadrados intra-cluster, e é útil reconhecer os três.

91 Por que a documentação diz que a inércia “não é uma métrica normalizada”? Que problema isso cria?

Porque seu valor depende da escala das variáveis, de N e de D , e não existe nenhum limite superior de referência. Um valor de inércia igual a 4 217 não diz nada isoladamente: pode ser excelente ou terrível dependendo do conjunto. O problema prático é duplo: não se pode comparar inércias entre conjuntos de dados diferentes, nem entre versões diferentes do mesmo conjunto (padronizado versus não padronizado); e não se pode usá-la para escolher K , porque ela decresce monotonicamente. A comparação legítima é entre execuções com o **mesmo K sobre os mesmos dados**, que é justamente o que o `n_init` faz. Para comparar valores de K , use silhueta ou, em modelos probabilísticos, BIC.

92 Cite as três suposições implícitas do K-Means e o que quebra em cada caso.

(1) **Clusters isotrópicos (esféricos)**. Quebra em dados anisotrópicos: grupos alongados são cortados transversalmente pelas fronteiras de Voronoi, porque a distância euclidiana não sabe que uma direção é mais “longa” que outra. (2) **Variâncias semelhantes**. Quebra com grupos de dispersões diferentes: o algoritmo fatia o grupo mais espalhado e funde os compactos, já que a inércia soma distâncias sem normalizar por cluster. (3) **Tamanhos semelhantes**. Quebra com grupos de contagens muito diferentes: as fronteiras equidistantes entre centróides não conseguem representar que um grupo tem dez vezes mais pontos que outro. As três falham simultaneamente no exemplo clássico da documentação, com blobs anisotrópicos e de variâncias desiguais.

93 Por que rodar PCA antes do K-Means alivia a maldição da dimensionalidade?

Em alta dimensão as distâncias euclidianas se concentram: a diferença entre a distância ao ponto mais próximo e ao mais distante encolhe em termos relativos, e “o centróide mais próximo” deixa de ser informação discriminante — as atribuições ficam quase arbitrárias. O PCA projeta em um subespaço de poucas dimensões que retém a maior parte da variância, onde as distâncias voltam a ter contraste. Dois ganhos colaterais: descartar componentes de baixa variância remove ruído, e reduzir D acelera diretamente o algoritmo, que é $O(NKD)$. A ressalva a mencionar em prova: o PCA maximiza variância, não separabilidade, então em casos raros ele descarta justamente a direção que separava os clusters.

94 O que faz `n_init` e por que o padrão não é 1?

Executa o algoritmo n vezes com inicializações diferentes e retorna a execução de **menor inércia**. O padrão não é 1 porque o K-Means converge para um mínimo local, e uma única execução pode cair num resultado ruim — especialmente se duas sementes caírem no mesmo grupo real. Rodar várias vezes amostra vários mínimos locais e fica com o melhor. Vale notar que, com `init='k-means++'`, a variância entre execuções já é bem menor do que com inicialização aleatória, e por isso as versões recentes do scikit-learn reduziram o valor padrão de `n_init` — as duas defesas se sobrepõem parcialmente.

95 Quando usar `MiniBatchKMeans` em vez de `KMeans` ?

Quando N é grande o bastante para que o tempo de execução seja o gargalo, e uma perda pequena de qualidade seja aceitável. O MiniBatch sorteia um lote a cada iteração, atribui apenas aquelas amostras e atualiza os centróides por média corrente com taxa decrescente — o custo por iteração deixa de depender de N . Ele converge em uma fração do tempo e produz uma inércia **ligeiramente pior**. Também é a opção natural para aprendizado em fluxo, via `partial_fit`, quando os dados não cabem em memória. Se N é moderado, não há razão para abrir mão da qualidade: use o `KMeans` padrão.

96 Explique o esquema de atualização do `MiniBatchKMeans`.

A cada iteração: (1) sorteia-se um mini-lote de tamanho fixo; (2) cada amostra do lote é atribuída ao centróide mais próximo; (3) cada centróide é atualizado como uma **média corrente** — desloca-se na direção das amostras recém-atribuídas, com um peso que decresce à medida que aquele centróide acumula amostras. Essa taxa decrescente é o que estabiliza o processo: no começo os centróides se movem bastante, e com o tempo as atualizações ficam cada vez menores, o que reduz o efeito estocástico dos lotes individuais. É a mesma ideia do EM sequencial do Bishop 15.4.5 aplicada ao caso-limite rígido, e a mesma família de algoritmos do gradiente estocástico.

97 Qual a diferença entre o paralelismo de `OMP_NUM_THREADS` e o de `n_init`?

`OMP_NUM_THREADS` controla o paralelismo de **baixo nível, dentro** de uma única execução: o cálculo das distâncias é dividido em blocos e distribuído entre threads via OpenMP no código Cython, o que além de paralelizar melhora o uso de cache. O `n_init` descreve quantas execuções **independentes** são feitas — um paralelismo de nível mais alto, entre tarefas que não compartilham estado. São ortogonais: você pode ter 8 threads acelerando cada uma de 10 inicializações. Na prática, atenção a um efeito comum: combinar paralelismo em dois níveis pode gerar *oversubscription* (mais threads que núcleos) e piorar o desempenho.

98 Você tem dois anéis concêntricos. Por que o K-Means falha e qual método usar?

Falha porque a estrutura é **não plana**: os dois anéis não são separáveis por nenhum hiperplano, e o K-Means só produz fronteiras de Voronoi, que são retas. Pior: o centróide do anel externo cai no centro geométrico da figura, dentro do anel interno, então nem os protótipos fazem sentido. Nenhum K , nenhuma inicialização e nenhum `n_init` resolve — é a hipótese que está errada, não o ajuste. Os métodos adequados são os de geometria não plana: **DBSCAN**, que agrupa por conectividade de densidade e separa os anéis naturalmente, ou **spectral clustering**, que projeta os dados num espaço onde a estrutura vira linearmente separável antes de agrupar. Um GMM também falharia, embora menos catastroficamente, porque sua geometria também é plana (elipsoidal).

99 Compare K-Means e DBSCAN.

K-Means: você informa K ; assume clusters convexos e isotrópicos; todo ponto recebe um cluster; usa distância ao centróide; escala muito bem; produz centróides interpretáveis. **DBSCAN**: você informa o raio da vizinhança e o número mínimo de pontos, não K ; encontra clusters de **forma arbitrária**; classifica pontos em regiões esparsas como **ruído**, sem forçar atribuição; usa conectividade por densidade; escala bem; não produz protótipos. Escolha o DBSCAN quando não souber quantos grupos existem, quando as formas forem irregulares ou quando houver outliers a isolar. Escolha o K-Means quando souber K , quando os grupos forem blobs e quando precisar de centróides. A fraqueza do DBSCAN é a densidade variável — grupos densos e esparsos no mesmo conjunto exigem HDBSCAN ou OPTICS.

100 Onde o GMM aparece na tabela comparativa e como ele é caracterizado?

Aparece com caso de uso “**estimação de densidade**, geometria plana”, métrica “distâncias de Mahalanobis”, escalabilidade “não escalável” e parâmetros “muitos”. Cada um desses campos é informativo. “Estimação de densidade” sinaliza que o GMM não é primariamente um algoritmo de clustering: agrupar é um subproduto de modelar $p(\mathbf{x})$. “Geometria plana” o coloca do mesmo lado do K-Means — ele é mais flexível (elipses em vez de esferas), mas ainda não captura anéis nem espirais. “Mahalanobis” é o que dá essa flexibilidade extra. “Não escalável” reflete o custo $O(NKD^2)$ das covariâncias completas.

101 Bisecting K-Means aparece como produzindo “clusters mais uniformes que o K-Means”. Por quê?

Porque ele constrói a solução **hierarquicamente**: começa com todos os pontos em um cluster e, repetidamente, escolhe um cluster e o divide em dois com um K-Means de $K = 2$, até chegar aos K desejados. Como a escolha de qual cluster dividir costuma recair sobre o de maior inércia ou maior número de pontos, o procedimento naturalmente evita deixar um cluster gigantesco ao lado de vários minúsculos — desequilíbrio que o K-Means padrão produz com facilidade. Benefícios adicionais: escala melhor para K grande (cada divisão é um problema pequeno) e produz uma árvore hierárquica que permite examinar a estrutura em várias granularidades.

102 A documentação diz que o K-Means divide as amostras em grupos de “variância igual”. Isso é hipótese ou resultado?

É **hipótese**, embutida na função objetivo — e a frase é uma descrição do que o algoritmo tenta fazer, não uma garantia sobre os dados. Ao minimizar a soma de quadrados sem normalizar por cluster, o método distribui a “energia” de dispersão de forma tendencialmente equilibrada entre os grupos, porque um cluster com variância muito maior contribui desproporcionalmente para a inércia total e é vantajoso dividi-lo. A consequência é que, se os grupos reais têm variâncias muito diferentes, o K-Means **não** os recupera: ele impõe a sua hipótese aos dados, fatiando o grupo disperso e fundindo os compactos. Reconhecer isso como hipótese, e não como propriedade dos dados, é o que permite prever o modo de falha.

103 Quais métodos da tabela dispensam informar o número de clusters?

DBSCAN, HDBSCAN, OPTICS, Mean-shift e Affinity propagation. Eles não pedem K , mas pedem **outros** hiperparâmetros que determinam indiretamente quantos clusters emergem: DBSCAN pede o raio da vizinhança ϵ e o mínimo de pontos; Mean-shift pede a largura de banda; Affinity propagation pede a preferência de amostra e o damping. Não é, portanto, uma dispensa de decisão — é uma **troca** de qual decisão você toma. A vantagem é que às vezes o parâmetro alternativo é mais fácil de justificar a partir do domínio (“dois pontos a menos de 5 metros são vizinhos” é mais concreto que “existem 7 grupos”).

104 Descreva o fluxo completo de uma análise de clustering bem feita.

(1) **Entender os dados** e o objetivo: o domínio é contínuo, binário, categórico? Quer-se partição, densidade ou detecção de anomalias? (2) **Pré-processar**: tratar faltantes, padronizar (obrigatório para métodos baseados em distância) e, se D for grande, reduzir dimensionalidade. (3) **Escolher o método pela hipótese geométrica**, usando a tabela do §2.3.1 — não pela popularidade. (4) **Varrer os hiperparâmetros**, tipicamente K , avaliando com silhueta, BIC ou verossimilhança de validação. (5) **Validar**: estabilidade por reamostragem, inspeção visual em projeção, e verificação de sentido com quem conhece o domínio. (6) **Interpretar**: caracterizar cada grupo pelos seus centróides ou perfis. (7) **Documentar** as escolhas e as limitações — sobretudo a hipótese geométrica assumida e o critério usado para K .

Simulado integrador

Questões que cruzam as quatro referências, no formato de arguição oral. Tente responder antes de ler.

As perguntas anteriores testam cada referência isoladamente. Estas testam se você consegue **ligar** as quatro — que é o que uma banca costuma fazer quando quer separar quem estudou de quem decorou.

SIMULADO · 12 QUESTÕES DE ARGUIÇÃO

S1 Conte a história completa que liga K-Means, mistura de Bernoulli e GMM em um parágrafo.

Todos os três resolvem o mesmo problema — descobrir estrutura de grupos sem rótulos — com a mesma arquitetura: uma variável latente discreta $\mathbf{z}$ em codificação 1-de- K que diz de qual componente cada amostra veio, e um procedimento de duas fases que alterna entre inferir $\mathbf{z}$ e reestimar os parâmetros. O que os separa é a **hipótese sobre** $p(\mathbf{x} | \mathbf{z})$. O K-Means não faz hipótese nenhuma: descarta a probabilidade, usa atribuição rígida e minimiza distância quadrada — e é, por isso, o caso-limite rígido do GMM com $\Sigma_k = \epsilon \mathbf{I}$ e $\epsilon \rightarrow 0$. A mistura de Bernoulli supõe D moedas independentes por componente, apropriada para $\{0, 1\}^D$. O GMM supõe gaussianas multivariadas, apropriadas para $\mathbb{R}^D$, e é o único que descreve forma e orientação via Σ_k . Os dois últimos são ajustados por EM, cuja correção decorre da decomposição $\ln p(\mathbf{X}) = \mathcal{L} + \text{KL}$.

S2 A mesma quantidade aparece com três nomes nas quatro referências. Quais e por quê?

Distorção (Bishop 15.1 e Murphy 21.3) e **inércia** (scikit-learn 2.3.2) são a mesma soma de quadrados intra-cluster, $J = \sum_n \min_k \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2$. Os nomes vêm de tradições diferentes: “distorção” é vocabulário de teoria da informação e quantização (o erro introduzido ao substituir um sinal por seu representante), “inércia” é vocabulário de estatística multivariada francesa (a dispersão de uma nuvem de pontos em torno do seu centro). Um terceiro nome que aparece na literatura é **within-cluster sum of squares (WCSS)**. Reconhecer os três é útil porque enunciados de prova podem usar qualquer um.

S3 Por que o K-Means precisa de heurísticas para escolher K enquanto o GMM tem o BIC?

Porque a seleção de modelo por critérios de informação exige uma **verossimilhança**, e o K-Means não define nenhuma distribuição de probabilidade. Sem $p(\mathbf{x} | \boldsymbol{\theta})$ não há $\hat{L}$, e sem $\hat{L}$ não há BIC nem AIC nem verossimilhança marginal. Resta o que se pode calcular sem modelo: a inércia (que decresce monotonicamente e por isso não serve sozinha, exigindo a heurística visual do cotovelo) e índices geométricos como a silhueta (que funcionam, mas medem separação convexa e são caros, $O(N^2)$). O GMM, sendo generativo, entra na maquinaria estatística padrão de seleção de modelos. É mais uma consequência da mesma diferença estrutural que dá ao GMM a detecção de anomalias e a geração de amostras.

S4 Ligue os três mecanismos de regularização que aparecem nas quatro referências. Eles são a mesma ideia?

Sim, são a mesma ideia em três disfarces, e Bishop 15.4.3 é onde ela aparece de forma explícita: **substituir a máxima verossimilhança pelo MAP, somando $\ln p(\boldsymbol{\theta})$ ao passo M**. (1) A **suavização de Laplace** na mistura de Bernoulli é o passo M com prior Beta, e impede μ_{ki} de travar em 0 ou 1. (2) O **reg_covar** do scikit-learn soma $\epsilon \mathbf{I}$ à covariância, o que aproxima o passo M com um prior sobre $\boldsymbol{\Sigma}$, e impede o colapso singular. (3) A **BayesianGaussianMixture** faz isso de forma completa e principiada, com priores sobre todos os parâmetros e inferência variacional. Em todos os casos o efeito é o mesmo: tornar inatingíveis as configurações degeneradas que a verossimilhança pura premia.

S5 Um colega diz: “rodei K-Means com K de 2 a 20 e escolhi $K = 20$ porque teve a menor inércia”. Corrija.

O raciocínio está errado por construção. A inércia é monotonicamente não-crescente em K : sempre é possível reduzi-la dividindo um cluster, e em $K = N$ ela vale exatamente zero. Portanto “o K de menor inércia” é sempre o maior K testado — o resultado não diz nada sobre os dados, apenas reflete a mecânica da métrica. É o mesmo erro de escolher a ordem de um polinômio pelo menor erro de treino. O que fazer: procurar o **cotovelo** da curva (heurística), maximizar a **silhueta** (que penaliza a perda de separação e por isso tem máximo genuíno), ou, se um modelo probabilístico for aceitável, ajustar um GMM e minimizar o **BIC**, que penaliza explicitamente a complexidade. E cruzar pelo menos dois desses critérios antes de decidir.

56 **Deduzo por que o passo E é “Bayes” e o passo M é “máxima verossimilhança ponderada”.**

O **passo E** pergunta: dado o que sei dos parâmetros, qual a probabilidade de esta amostra ter vindo do componente k ? Isso é literalmente um posterior, $p(z_k = 1 \mid \mathbf{x})$, calculado por Bayes com prior π_k , verossimilhança $p(\mathbf{x} \mid \theta_k)$ e a evidência no denominador. O **passo M** pergunta: dadas essas atribuições (fracionárias), quais parâmetros melhor explicam os dados? Se as atribuições fossem rígidas, a resposta seria a estimativa de máxima verossimilhança usual de cada componente sobre o seu subconjunto. Como são fracionárias, cada amostra entra em cada componente com peso γ_{nk} , e o que se obtém são as mesmas fórmulas de MLE com somas ponderadas e N_k no lugar de N . Em resumo: **E infere as latentes por Bayes; M ajusta os parâmetros por MLE, ponderado pelo resultado de E.**

57 **Você tem 50 000 documentos representados por um vetor binário de 5 000 palavras. Qual modelo usar e por quê?**

Mistura de Bernoulli, pelo domínio: os dados são $\{0, 1\}^{5000}$ e a gaussiana seria um erro de modelagem. Considerações de execução: (i) o EM tem $KD + K - 1$ parâmetros — com $K = 20$, cerca de 100 mil, viável com 50 mil documentos, mas justo, então use suavização de Laplace; (ii) o subfluxo é **certo**, pois o produto de 5 000 fatores menores que 1 colapsa em `float64` — trabalhe em espaço logarítmico com Log-Sum-Exp desde o início, não como otimização posterior; (iii) inicialize com k-means++ sobre a distância de Hamming ou com pequenas perturbações da média global; (iv) escolha K por verossimilhança em um conjunto de validação separado. Alternativa a mencionar: se os documentos tiverem contagens em vez de indicadores binários, uma mistura multinomial ou o LDA seriam mais apropriados.

58 **Explique por que a log-verossimilhança nunca diminui, sem usar a palavra “ELBO”.**

Imagine uma curva, a log-verossimilhança em função dos parâmetros, e uma segunda curva que fica sempre **abaixo** dela, mas que podemos construir de modo a **tocá-la** num ponto à nossa escolha. O passo E constrói essa curva auxiliar de forma que ela toque a log-verossimilhança exatamente no θ atual. O passo M então escala a curva auxiliar — ele acha o máximo *dela*, não da original. Como a auxiliar nunca está acima da original, o valor da original no novo ponto é pelo menos o valor da auxiliar ali, que por sua vez é pelo menos o valor da auxiliar no ponto de partida, que é *igual* ao valor da original no ponto de partida. Encadeando: a original no ponto novo $\geq$ a original no ponto antigo. Nunca desce. (E a diferença entre as duas curvas no ponto novo é a KL que reabriu — é o ganho extra.)

S9 Compare os modos de falha numérica do GMM e da mistura de Bernoulli.

Falham por motivos **opostos**. O GMM falha por **explosão**: um componente colapsa sobre um ponto, $|\Sigma_k| \rightarrow 0$, a densidade e a log-verossimilhança divergem para $+\infty$. É uma patologia da *função objetivo*, que é genuinamente ilimitada — a defesa é regularizar a covariância (`reg_covar`) ou usar priores. A Bernoulli falha por **desaparecimento**: o produto de D probabilidades subflui para zero em ponto flutuante, e a razão do passo E vira $0/0$. É uma patologia da *aritmética de ponto flutuante*, não da matemática — em precisão infinita não haveria problema, e a defesa é o Log-Sum-Exp, que é uma identidade exata. Há ainda o travamento em $\mu_{ki} \in \{0, 1\}$, que é problema de estimação e se resolve com suavização de Laplace. Regra mnemônica: **GMM explode por causa da geometria; Bernoulli some por causa dos bits**.

S10 Em que ordem você apresentaria estes conceitos a alguém que nunca os viu, e por quê?

Na ordem que Bishop e Murphy adotam, que é a mesma: **K-Means primeiro**. Ele é visual, tem duas regras simples, converge à vista e não exige nenhuma probabilidade — cria a intuição de “protótipo” e de “alternar entre atribuir e atualizar” que todo o resto reaproveita. Em seguida, mostre **onde ele falha** (anisotropia, densidades desiguais, dados binários): é a motivação genuína para o que vem depois, e sem ela os modelos de mistura parecem complicação gratuita. Aí introduza a **variável latente** e a **responsabilidade**, que generalizam a atribuição rígida para suave. Depois o **EM** como o algoritmo que substitui o Lloyd, e só então a **prova pelo ELBO**. Fechar revelando que o K-Means era o caso-limite do EM o tempo todo dá ao aluno a sensação de que o arco se fecha — e é literalmente verdade.

S11 Qual a diferença entre “o EM convergiu” e “o EM encontrou a resposta certa”?

Convergir significa apenas que a log-verossimilhança parou de subir de forma apreciável — que se chegou a um ponto estacionário. Isso não garante três coisas distintas. (1) Não garante o **máximo global**: pode ser um máximo local, ou até uma sela; a superfície de verossimilhança de misturas é notoriamente multimodal. (2) Não garante que o **modelo** esteja certo: um GMM converge lindamente sobre dois anéis concêntricos e produz uma resposta sem sentido, porque a hipótese geométrica é inadequada — o algoritmo não tem como detectar isso. (3) Não garante que K esteja certo. Convergência é uma propriedade do *otimizador*; adequação é uma propriedade do *modelo*. Verificar a segunda exige inspeção, validação e conhecimento do domínio — nunca o log de convergência.

S12 Se você pudesse guardar apenas cinco ideias de todo este conteúdo, quais seriam?

(1) **A variável latente discreta é o conceito unificador**: os três modelos são a mesma estrutura com hipóteses diferentes sobre $p(\mathbf{x} | \mathbf{z})$. (2) **A responsabilidade γ_{nk} é o teorema de Bayes** aplicado a essa latente, e é o que transforma atribuição rígida em suave. (3) **O EM existe porque o somatório está preso dentro do logaritmo**; ele contorna isso otimizando a verossimilhança completa em esperança, e a decomposição $\ln p = \mathcal{L} + \text{KL}$ prova que isso nunca piora. (4) **K-Means é o caso-limite rígido do GMM** — a relação não é analógica, é exata, com $\Sigma_k = \epsilon \mathbf{I}$ e $\epsilon \rightarrow 0$. (5) **Todo método traz uma hipótese geométrica**, e escolher o método é escolher a hipótese: nenhum ajuste de hiperparâmetro conserta a hipótese errada.

Formulário

Todas as equações essenciais reunidas, para revisão de véspera.

A.1 K-Means

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2 = \sum_{n=1}^N \min_k \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2$$

$$r_{nk} = 1 \left[k = \arg \min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 \right] \quad \boldsymbol{\mu}_k = \frac{\sum_n r_{nk} \mathbf{x}_n}{\sum_n r_{nk}}$$

Custo $O(NKD)$ por iteração. Converge para mínimo local. Fronteiras de Voronoi (hiperplanos).

A.2 k-means++ e silhueta

$$p(\mathbf{x}) = \frac{D(\mathbf{x})^2}{\sum_n D(\mathbf{x}_n)^2}, \quad D(\mathbf{x}) = \min_j \|\mathbf{x} - \boldsymbol{\mu}_j\| \quad \mathbb{E}[J_{++}] \leq 8(\ln K + 2) J_{\text{ótimo}}$$

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1]$$

A.3 Gaussian multivariada e GMM

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2} |\boldsymbol{\Sigma}|^{1/2}} \exp\left\{-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right\}$$

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad \sum_k \pi_k = 1, \quad \pi_k \geq 0$$

$$\ln p(\mathbf{X}) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

A.4 EM para GMM

$$\text{Passo E: } \gamma(z_{nk}) = \frac{\pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} \quad N_k = \sum_{n=1}^N \gamma(z_{nk})$$

$$\text{Passo M: } \boldsymbol{\mu}_k = \frac{1}{N_k} \sum_n \gamma(z_{nk}) \mathbf{x}_n, \quad \pi_k = \frac{N_k}{N}$$

$$\boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_n \gamma(z_{nk}) (\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^\top$$

A.5 Mistura de Bernoulli

$$p(\mathbf{x} \mid \boldsymbol{\mu}_k) = \prod_{i=1}^D \mu_{ki}^{x_i} (1 - \mu_{ki})^{1-x_i} \quad p(\mathbf{x}) = \sum_{k=1}^K \pi_k p(\mathbf{x} \mid \boldsymbol{\mu}_k)$$

$$\gamma(z_{nk}) = \frac{\pi_k p(\mathbf{x}_n \mid \boldsymbol{\mu}_k)}{\sum_j \pi_j p(\mathbf{x}_n \mid \boldsymbol{\mu}_j)} \quad \boldsymbol{\mu}_k = \frac{1}{N_k} \sum_n \gamma(z_{nk}) \mathbf{x}_n \quad \pi_k = \frac{N_k}{N}$$

Laplace: $\mu_{ki} = \frac{\sum_n \gamma(z_{nk}) x_{ni} + \alpha}{N_k + 2\alpha}$ LSE: $\ln \sum_k e^{a_k} = a_{\max} + \ln \sum_k e^{a_k - a_{\max}}$

A.6 ELBO e a relação K-Means ↔ GMM

$$\ln p(\mathbf{X} \mid \boldsymbol{\theta}) = \mathcal{L}(q, \boldsymbol{\theta}) + \text{KL}(q \parallel p), \quad \text{KL} \geq 0 \Rightarrow \mathcal{L} \leq \ln p(\mathbf{X} \mid \boldsymbol{\theta})$$

$$\mathcal{L}(q, \boldsymbol{\theta}) = \sum_{\mathbf{Z}} q(\mathbf{Z}) \ln \frac{p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta})}{q(\mathbf{Z})} \quad \mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}}) = \sum_{\mathbf{Z}} p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta}^{\text{old}}) \ln p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta})$$

$$\boldsymbol{\Sigma}_k = \epsilon \mathbf{I}, \quad \pi_k = 1/K, \quad \epsilon \rightarrow 0 \quad \Rightarrow \quad \gamma(z_{nk}) \rightarrow r_{nk} \in \{0, 1\} \quad \text{e} \quad \text{EM} \rightarrow \text{K-Means}$$

A.7 Seleção de modelo e contagem de parâmetros

$$\text{BIC} = -2 \ln \hat{L} + p \ln N \quad \text{AIC} = -2 \ln \hat{L} + 2p \quad (\text{menor é melhor})$$

MODELO	PARÂMETROS LIVRES P
GMM <code>full</code>	$(K-1) + KD + K D(D+1)/2$
GMM <code>tied</code>	$(K-1) + KD + D(D+1)/2$
GMM <code>diag</code>	$(K-1) + KD + KD$
GMM <code>spherical</code>	$(K-1) + KD + K$
Mistura de Bernoulli	$(K-1) + KD$

A.8 Quadro comparativo final

	K-MEANS	MISTURA DE BERNOULLI	GMM
Domínio	$\mathbb{R}^D$	$\{0, 1\}^D$	$\mathbb{R}^D$
Hipótese	Hiperesferas isotrópicas	D Bernoullis independentes	Elipsóides $\mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$
Atribuição	Rígida, $r_{nk} \in \{0, 1\}$	Suave, $\gamma_{nk} \in [0, 1]$	Suave, $\gamma_{nk} \in [0, 1]$
Critério	Minimizar J	Maximizar $\ln p(\mathbf{X})$	Maximizar $\ln p(\mathbf{X})$
Algoritmo	Lloyd	EM	EM
Custo/iteração	$O(NKD)$	$O(NKD)$	$O(NKD^2)$

Generativo?	Não	Sim	Sim
Escolha de K	Cotovelo, silhueta	BIC, AIC, validação	BIC, AIC, validação
Modo de falha	Mínimo local; anisotropia	Subfluxo; $\mu \rightarrow \{0, 1\}$	Singularidade $ \Sigma \rightarrow 0$

SOBRE AS FONTES DESTE GUIA

O conteúdo foi redigido a partir do material canônico das quatro referências indicadas e da apresentação do grupo. Os textos originais não puderam ser consultados diretamente durante a redação, porque o ambiente de rede utilizado bloqueia o acesso a `bishopbook.com`, `probm1.github.io` e `scikit-learn.org`. As definições, equações e resultados apresentados são padrão da literatura e conferem com os livros; a **numeração exata das subseções**, porém, deve ser verificada nos sumários originais antes de ser citada em trabalho avaliado.